Faculty of Informatics and Computer Science

Software Engineering

Sales Forecasting Project using machine learning

By: Youssef Ahmad

ID: 207845

Supervised By

Dr Mohamed Mead

November 2023

Abstract

The problem in this time is that economic are changing from year to year and in most cases, it is changing to the worse; so because this issue it gave me the motivation to use the powerful capabilities of deep learning models such as LSTM and ARIMA. The reason behind selecting these models is that these models are tailored to be used with time series analysis. And it will be integrated with a website based on MERN Stack to implement a business aid tool to help businesses owners manage their day-to-day operations. Expected outcome of the project is a fully functional full-stack application with data handling operations (CRUD) and visualization tools to collect important da-ta and display it in a good way to make business decisions, and improve overall performance of the business

Attestation &Turnitin Report

I understand the nature of plagiarism, and I am aware of the University's policy on this.

I certify that this report reports original work by me during my University project.

Signature **Date**

8/6/2024

Acknowledgements

I cannot find enough words to thank my supervisor Dr. Mohammed Mead for his continuous support and encouragement. For him I give my sincere appreciation for the learning opportunities I found through working under his supervision through this year.

I would like also to Acknowledge and thank the TA's who were always available for my help when I needed, special thanks to Engy Samir.

Finally, to my loving, caring family, I would like to express my love and deepest gratitude.

Contents

List of Figures

1 Introduction

Retail industry is when goods and items are bought from manufactures, then selling these goods for customers with a higher price to gain profits. Retail industry tasks involve managing how the goods are ordered , sent to stores, and paid for the goods. They also focus on keeping the stores as organized as possible. While also focusing on having a good relationship with the customers as having loyalty customers is important for any retail business. However, in the latest years retailers have faced a lot of problems. Firstly, now the market is changing, Milennials are now the majority of their customers, Millennials now who are used to internet for most of their lives. Selling products to Millennials is difficult as you need to know the products that will engage them with you and what shopping experience they are looking for. While also making sure you still have the older generations loyal to you. At the same time retailers are not certain about the future as it is changing and the world economy is changing rapidly, and now it is important for business owners to predict customers' trends and demand to stay ahead of your competitors. However, despite all the challenges the retailers have technology is now providing them with a handful of tools to face these challenges and even use them for their advantage. Technologies that are provided for them are focused on the big data analysis field. As it uses the data that these retailers have about the customers, sales, times where sales rise and increases, using all these data with the right tool will help all retailers in overcoming all these challenges by helping them predict trends and demand [1].

1.1 Overview

One of the most important tools that retailers can use to gain a competitive advantage over other retailers is by using sales and demand forecasting tools.

Retail sales forecasting focuses on generating forecasts for large amount of data regarding products that is being sold across several stores. Now sales forecasting is the most important input for managerial decisions, like pricing their products, allocation of stores spaces, inventory management. sales forecasting also provide an estimation of sales quantity as SKU (Stock Keeping Unit) at store level, that should improve customers experience, reduce wastes of items in inventory, it can increase revenue for these retailers, and more efficient distribution [2]. Retail products demand is driven by many factors such as changes in price, promotions, sales, special days, and holidays also can heavily change the demand, and weather can also change that extremely cold or hot days people tend to stay at home, so this also affect the demand for products. Due to these reasons sales at stores goes through significant changes and it is not balanced [3]. Existing research in retail sales forecasting has tried to find one universal forecasting method for all types of products out there. Free-lunch theorem tells us there is guarantee that any complex forecasting method no matter how powerful and strong it is for a set of series than another method that is solely focuses in these series; implying that it not likely that one forecasting method can outperform other methods for all products and all future time periods [4]. Retail sales performance is specific to the application where it is used with, having unique features about the products which does not correspond with the massive competition studies [5]. This

makes the importance of method selection to deal with each problem there is not a lot of studies that has covered this point but there is a study that covered this part. F&P study found accuracy benefits when choosing a range of methods to match data and series forecasting performance bases on the features of the data and the type of products that the forecasts is done for. There may be benefits to have more than one method while forecasting for retailers to capture the features of the retail data [6]. Choosing the right forecasting method or methods for your retail store can increase your sales, help you manage your business and help you with organizing your inventory and keep improving your business.

Machine Learning

Any machine learning model has three features, firstly what should the model do, secondly what will the performance of the model be based on, and finally how will the model improve and get experience. For example, in a game of checkers the model has these 3 features it should be able to play the game; it should count how many times the model wins and how will that model improve. The generic model of any machine learning has 6 components no matter what it should achieve [7].

Machine Learning components

i. **Collect and prepare data:** the first step is being able to collect the needed data to achieve the goal of the model. These data have to be changed to an input so it can be then inserted into the algorithm, however the data is filled with noise (duplicated data or missing values) most of the time which will probably make the model come out with wrong answers or not an accurate outcome in the end; so the data should be cleaned and the missing data should be handled with carefully [8].

ii. **Feature Selection:** The collected data from the previous step is enormous most of the time with a lot of features that will not help the model in most cases it will distract the model from giving you a correct answer or an accurate answer. So unnecessary features should be removed from the data [8].

iii. **Choice of Algorithm:** There is a big amount of machine learning algorithms and not all of them are the best match for every problem out there. Some algorithms are suited for specific types of problems. Hence picking the right algorithm for the right problem is an essential part of building a powerful model [8].

iv. **Selection of Models and parameters:** many of the algorithms out there for machine learning need some manual modifications to get the most accurate values of various parameters [8].

v. **Training:** during the training process the algorithm feeds the model with the collected data and see the outcome of the model, then it should be compared with the real value in the data set that will help in optimization the model performance as modifications will be made depending on the model answers [8].

vi. **Performance Evaluation:** Last step that should be done in implementing the model is by testing the model with unseen data and collect various values like how accurate the model and the precision of the model is [8].

One of the machine learning algorithm that will be used when deploying a sales forecasting model will be a linear regression algorithm.

Linear Regression: it is argued that linear regression is the simplest ML Algorithm out there. Its main goal is to find a relationship between one or more numeric features into one numerical outcome. It is an analysis technique that is used to a regression problem by using a straight line to describe the dataset. For solving a linear regression problem that has one feature only it will have this form y = ax + b on an intercept slope [9, 10]. In that slope it will show how the y-axis increases when x increases in value, a indicates the rate at which y changes in aspect to x, and b is the y-intercept representing y value when x is 0. The task of the algorithm is finding the most optimal values of a and b to find the line that is the best at fitting the data points, which allows the model to make accurate predictions for new unseen data points [9,10]. As you can see in figure 1.

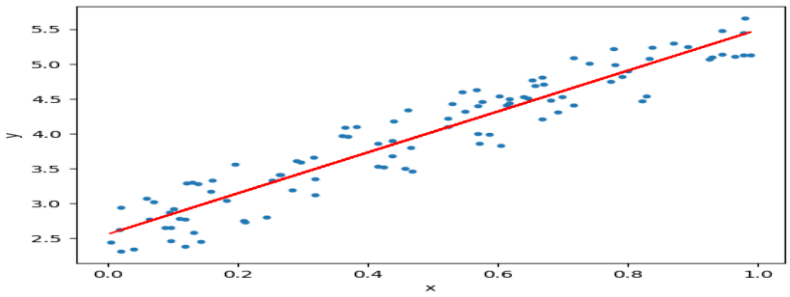


Figure 1: linear regression

Neural Networks and Deep Learning

An artificial neural network (ANN) is a machine learning algorithm that is inspired by biological neural networks [9, 10, 11]. In every ANN it contains a node that is capable of communication with other nodes through a connection. These connections are weighted upon whether they can or cannot provide a desired outcome [9, 10 ,11].

Feedforward Neural Networks

it uses perceptron which is an ML Algorithm that takes features and its targets that is trying to find a line, plane or a hyperplane that is separating these classes into two, three, or hyper dimensional space [9,12,13]. The sigmoid function is responsible for transforming these features (fig 7A). It is like logistic regression, but it provides the association of these classes not the probability that something is belonging to a specific class.

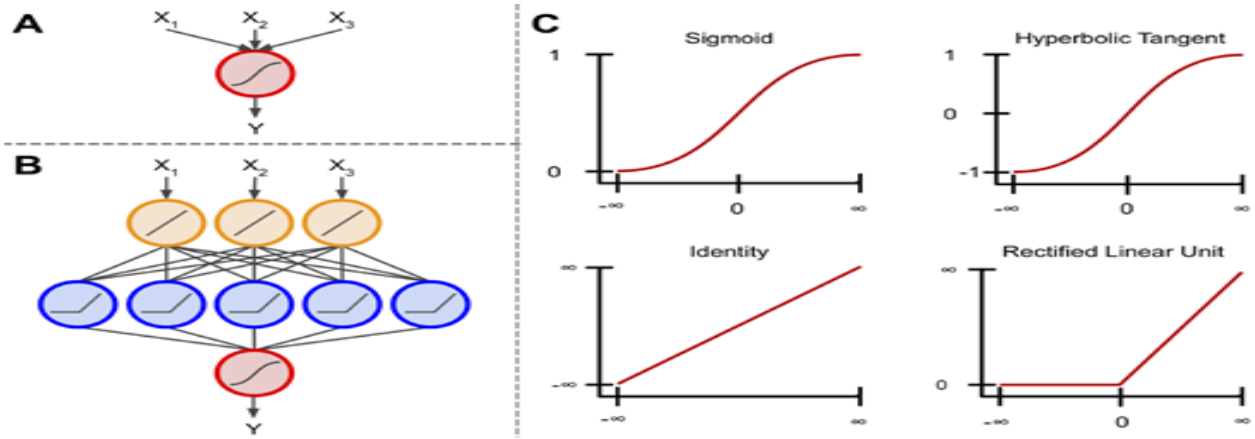


Figure 2: neural network structure and activation functions.

Components of a neural network. (1A) The foundation of Artificial neural networks, perceptron using the sigmoid function to transform more than one input into one output ranging from 0 to 1. (1B) Artificial neural networks connect multiple perceptron to use the input in getting the output, but it passes that output into and input to another.

When more than one perceptron's is connected than that model is concerned as a multiple perceptron network or ANN. Most ANNs have an input layer, output layer and hidden layers [9]. In simple ANNs the number of hidden layers is between 0 and 3, but in deep neural networks it consists of hundreds or even thousands of hidden layers [9,14]. ANN job is to pass information from one node to another, then it should be passed in all nodes at the first layer into the nodes of the second layer and keep forwarding until it goes through all the layers (Fig. 1B).

The number of nodes is not limited in ANN, however in the output layer should be corresponding to the number of classes that is being predicted in the case of classification of multi classes there should be one signal node in the output with a sigmoid function, there should be a single node with binary classification it should have one node with a linear function if the goal is regression [9, 14]. The activation functions (Fig. 1C) aim to change the node input into the desired output. Each node in ADD has its own activation function.

1.2 Problem Statement

It is easy to say now that the world is changing daily and for this reason it is hard to optimally manage your business, as the needs of the customers changes daily, and the customer needs are always changing. This makes it hard for all business owners to make correct decisions to improve their business, increase their profits and make predictions regarding what does the customers need. All these reasons are making it extremely difficult for all business owners in staying in the market. For these reasons business owners will need help by utilizing various machine learning techniques to build for them a sales forecasting model that can predict the needs of the customers and can help them optimally manage their business.

1.3 Scope and Objectives

- **Collect and prepare data**
  - Gather relevant and big data sources, including holiday calendars, fuel prices, temperature data, sales data with these parameters.
  - Clean the data to ensure that all the data that we have can be used in training a good model that gives us accurate data.
- **Data Visualization**
  - Build a way to easily visualize the data using charts, graphs, tables whatever needed to present the data to the user.
  - It should have a good-looking dashboard that can present the data in one place, and it should be easy to understand what this data represents.
- **Data Analysis and prediction**
  - The system should have all the latest and necessary algorithms to build a strong and accurate sales forecasting model that help the user to improve their business plan.
- **Feature select and Engineering**
  - Keep only the important features that allows us to give the user accurate predictions of the data and remove unnecessary features from the data that we have collected, to improve the model.

- **Simple User interface**
    - An easy-to-use user interface that should be easy to navigate and should be visually appealing to the eye.
    - It should not need excessive training to get used to using it.
- **Optimization and Scalability**
    - The model should be optimized to handle large datasets and to give us accurate sales predictions.

Focusing on these key features the AI tool will give various startup business owners various functionalities to empower them and hopefully help them getting over different businesses, by giving the users of this tool a way to help them daily manage their business on daily basis providing them with short-term planning and long-term planning to assist the business in improving. With the friendly user interface, and necessary functionalities the tool will be helpful to all startups.

1.4 Report Organization (Structure)

The rest of the report is organized as follows section 2 is a literature reviews for other papers that is talking about sales forecasting and demand forecasting. Section 3 is the used methodology to implement the model and the requirements of the system. Section 4 is what have I achieved in my project and what changes I made to the project.

1.5 Work Methodology

Develop a machine learning predictive model using neural networks algorithms for startup business owners, then a user-friendly website will be built which have features like visualization business data while making sure that is it presented to the user in a friendly way where it only shows all the business important data .Integrating the model into the website will help in improving the decision-making process for the business owner decisions that will help in managing company resources (Human resources, financial resources) it will improve the company overall profits. So, the purpose of this project is to provide a quick easy to understand tool that can help any business owner out there.

Tools: Java script frameworks like react, Express and node will be used in implementing the website that will host the Machine learning model.

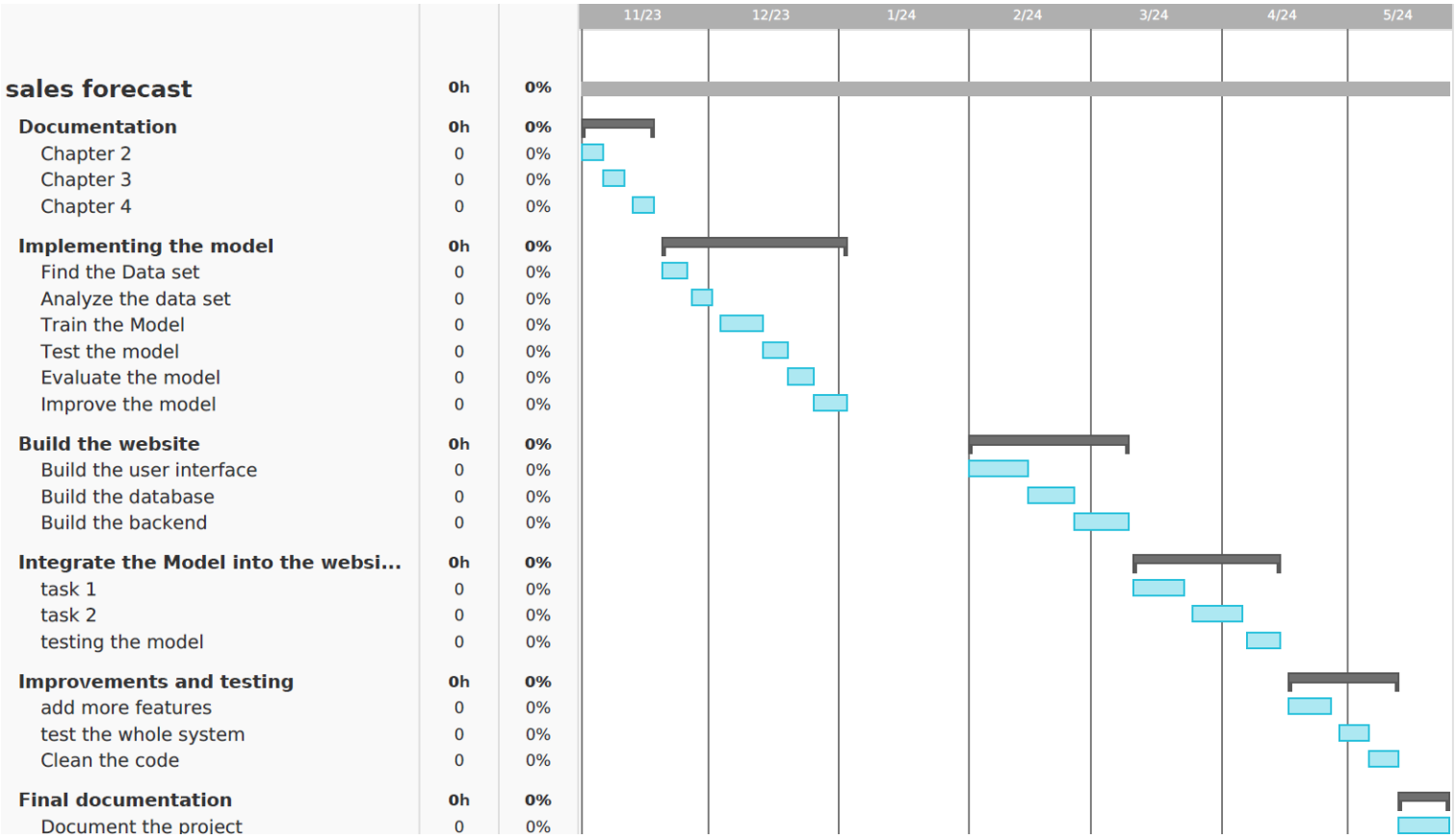
Mongo Database will be used to implement the Database of the website.

Python will be used to develop the model and I will use these libraries.

- scikit-learn to help with clustering, classification, and regression.

- tenserflow to build neural networks.

1.6 Work Plan (Gantt Chart)



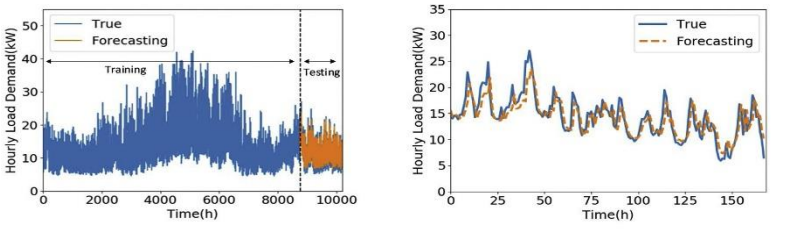
2 Related Work (State-of-The-Art)

2.1 Background

I have collected 12 research papers that is focus on implementing sales forecasting and demand forecasting models using various machine learning, neural networks, and deep learning algorithms. Each paper explained the algorithm they choose and why they choose it, the advantages, and disadvantages of each algorithm. They used sales forecasting tools for various businesses, but the results were the same with all businesses, however the chosen algorithm will improve the accuracy of the model and gives better results.

2.2 Literature Survey

- In this paper the proposed model is for forecasting electricity consumption of residential buildings. Several data-driven methods can be used to implement this model such ass Artificial neural networks, Statical regression, Decision Tree, and genetic algorithms all different algorithms have been used for forecasting residential buildings electricity. In this paper a deep learning approach to forecast load demand using an RNN model with GRU (DRNN-GRU). The model with evaluated using root mean square error (RMSE), mean absolute error (MAE), Pearson correlation coefficient (PCC), and mean absolute percentage error (MAPE). The Model achieves higher accuracy when compared to other models, it is also effective when there is missing historical data figure 5 below shows the performance of the model. However, the model needs to know the weather data to give accurate predictions if the weather data is incorrect or not accurate [15].



(a) Training and testing processes. (b) Segment forecasting results.

Figure 3: model results

- In this paper the proposed model is to deploy a forecasting model in supply chain management. This paper considered and LSTM (Long-Short Term Memory) model and a bidirectional BLSTM which can capture more complex patterns in time series using a real data to evaluate how it will improve the generated value generated by supply chain.

The LSTM and BLSTM was trained with time series customer demand for 10 specific products in the market. The data is for 3 years 2015, 2016, 2017 each time series had time series for monthly sales the length of the data is equal to 35. An extra step of power transformations for variance stabilization, although it may change the non-linear in time series, but such a step is required to allow the deep learning algorithm to capture dynamics in the data and learn about the trends of the products. Otherwise, it may be hard to capture these data in such a short-length time series. Both LSTM and BLSTM have MASE less than one, both algorithms did not over fit the training data set. BLSTM algorithms have better performance than normal LSTM, BLSTM is appeared to be better in demand forecasting. LSTM is a competitive method that have been able to outperform other methods in the forecasting field, BLSTM ability to process data in both directions improves the results of a normal LSTM model [16].

- This paper is proposing a model for retail supply chain demand. The paper proposed a novel cross-temporal forecasting framework (CTFF) to generate forecasts at all levels of retail supply chain. A deep learning method long-short term memory is being used as the forecasting method for the CTFF. The model is evaluated using multi-channel retain supply chain. With performance metrics and statical tests it has been observed that the CTFF is significantly better than direct forecasts, and it also has been observed that a bottom-up approach is better than a top-down approach. "*When two conditions are satisfied (1) the point-of-sales data is available, and (2) the demand patterns are similar for bottom level nodes, the bottom-up approach performs better than top-down approach*" (Gross & Sohl, 1990; Kahn, 1998; Williams & Waller, 2011) . Their approach is the concept of temporal hierarchies where it generates the horizon forecast for each node then it aggregate quarter, semi-annually, and annually sales. Implementation the model using LSTM networks with keras library; the data have been normalized in range -1 to 1 then creating lag variables the time-series data have changed into a supervised learning, the input was transformed into a 3d vector which is required when dealing with keras library with form of [samples, timestamps, and features]. After changing the data into that format, a training of the model happened using LSTM stateful layers, regularization have been performed to prevent overfitting and help improve the model generalization (Gal & Ghahramani, 2016). The configuration of the model was as follows dense layers, units {4,1}, activation function relu and sigmoid, optimizer adam, loss_metric mae, mse, max epoch 100 to implement the LSTM Network. The model was evaluated with ARME, ARSME and ARMAPE for direct forecasts and temporary reconciled forecasts. The data proved that the temporary reconciled forecasts (Where it tries to optimize the prediction, so it aligns with other forecasts for example to make

sure the weekly predictions align well with monthly predictions) have better results when compared to direct forecasts (Where it only gets the monthly predictions without considering relationships with the monthly values). The ARMSE of temporary reconciled forecasts is less than of direct forecasts which mean that it is have less uncertainty than direct forecasts. After collecting all direct level temporally forecast then a cross-sectional hierarchy bottom-up approach it then generates the parent's nodes as its clear in figure 6 below. Using the proposed framework, it outperformed all other direct forecasts across three metrices ARMAE, ARMSE and ARMPE, the results proved that the superiority of the framework over other direct forecasts [17].

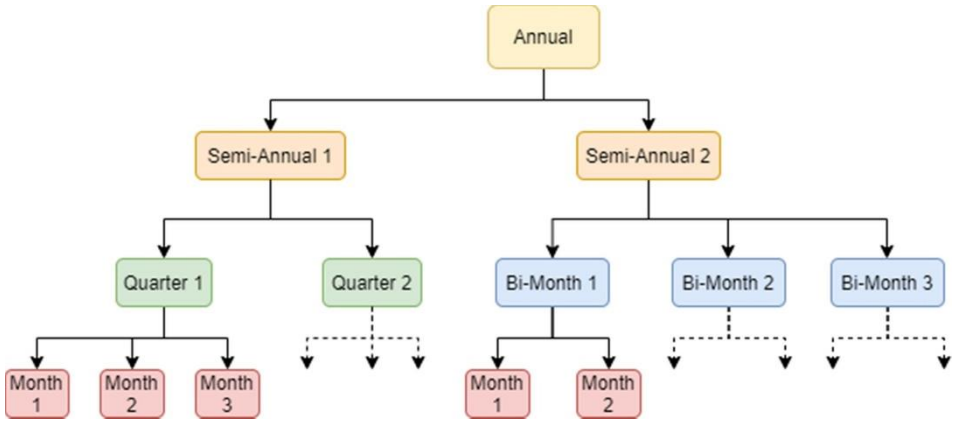


Figure 4: temporal hierarchies

- This paper proposed a demand forecasting method using LSTM network, it is a multi-layer LSTM network that is using the grid search approach. Their proposed method searches for the optimal hyperparameters for the LSTM network. One of their main advantages that their model have is the ability to find non-linear patterns in time series data. Their proposed model is applied for a real-word demand data of a furniture company, and they have compared it with other forecasting techniques. Their proposed model is in figure 5 below.

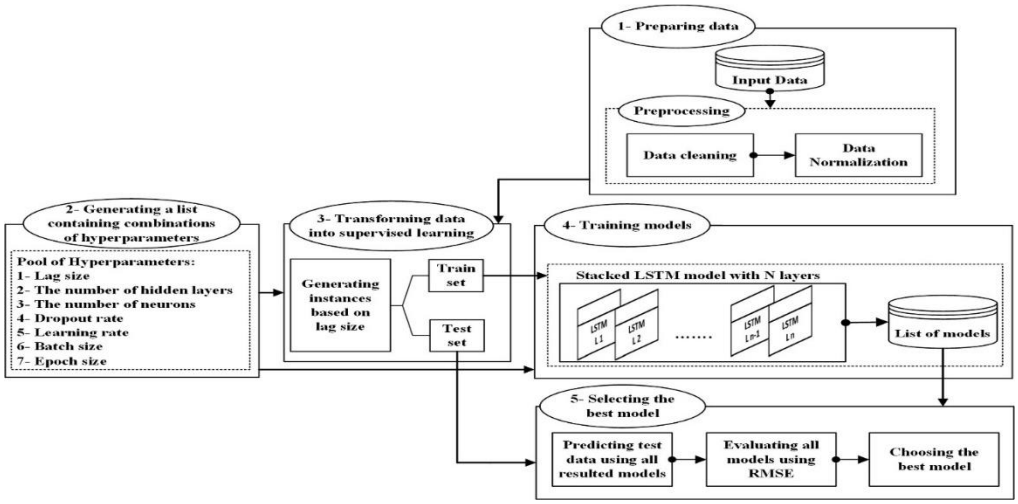


Figure 5: optimized hyperparameters model

"*LSTM network has achieved acceptable performance when applied on sequence data*" (Reimers & Gurevych, 2017). "*However, obtaining good performance with LSTM networks is not a simple task, as it involves the optimization of multiple hyperparameters*" (Reimers & Gurevych, 2017). In their study they customized 7 hyperparameters (Lag Size, number of hidden layers, number of neurons, dropout rate, Learning rate, batch size, and Epoch Size). Their method compared to a single layer LSTM as their method uses double hidden LSTM layer their method outperformed single layer LSTM in both RMSE and SMAPE tests as it improved generalization in their method and it utilized dropout process to fourthly enhance the model generalization. Their model compared to SVM it outperformed it in terms of RMSE and SMAPE, however SVM was the second most performing technique among all other methods. This study demonstrated the good performance of SVM in time series forecasting [18].

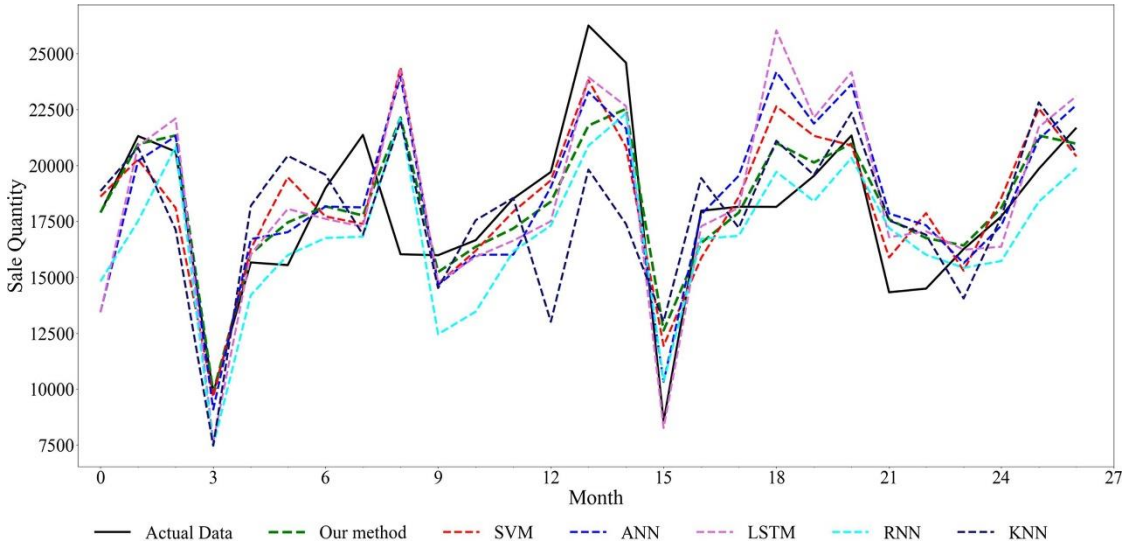
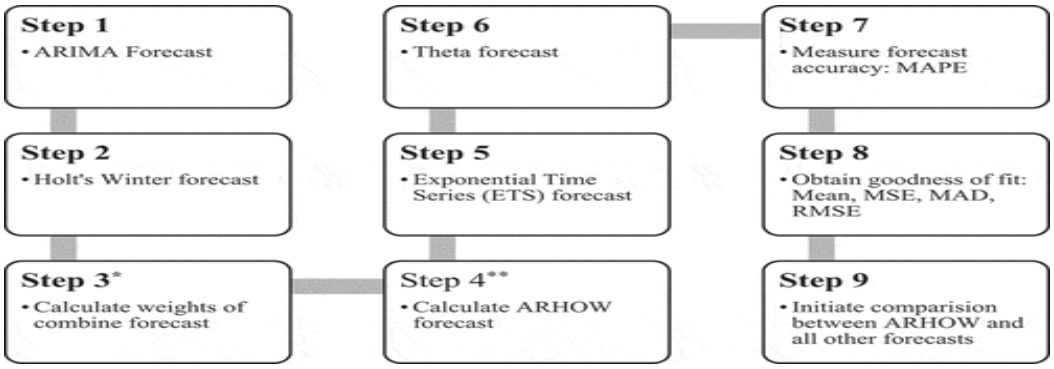


Figure 6: optimized hyperparameters model results

In figure 6 it shows the results of their proposed model compared with other various forecasting algorithms.

1. In this study they have proposed a new hybrid forecasting model that is combining



ARIMA (Autoregressive Integrated Moving Average) and Holt model referred to AR-HOW Model to use to forecast sales in the pharmaceutical industry. In figure 7 below it shows the structure of the algorithm and the steps that will be taken to build their model

Figure 7: ARHOW proposed Model

To test validity how efficient the ARHOW model is compared to ARIMA model, holt-winter, ETS and Theta forecast it have proven that it is significantly better compared to these models.

GETOFIN	MEAN	MAD	MSE	RMSE	MAPE
Sales in Unit	798				
ARIMA Forecast	*837*	*39*	*1,883*	*43*	5.64
Holt's Winter Forecast	859	66	5,414	74	9.37
ETS Forecast	842	44	2,376	49	6.33
Theta Forecast	*841*	*43*	*2,285*	*48*	6.21
ARHOW Forecast	*796*	*16*	*334*	*18*	1.99
Naïve Forecast	*808*	*65*	*7,650*	*87*	9

Figure 8: ARHOW proposed Model results compared to other models

This study has proven how important it is to combine two algorithms together as it will drastically improve the accuracy of the model and gives farther better results [19].

2. This paper has proposed an SFOR-ELM to build a sales forecasting model for online e-commerce websites to predict their sales and help them have a good business plan. The SFOR algorithm was proposed for the field for sales forecasting, combined with a data mining technique the SFOR-ELM Have been constructed to predict sales of clothes. The data used with the model is for an e-commerce website in China and the data have been collected from 2016 to 2019. The proposed model is shown in figure 9 below.

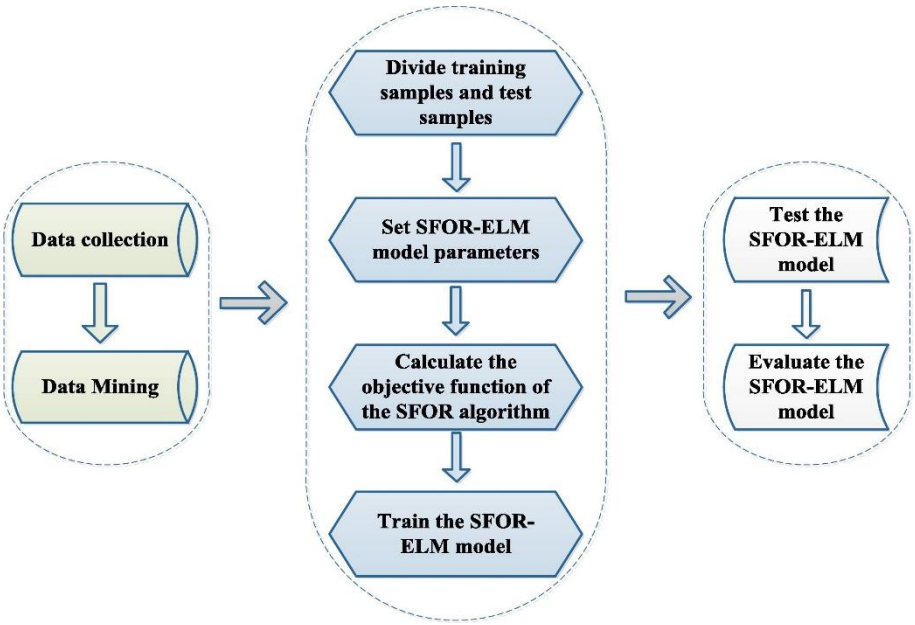


Figure 9: SFOR-ELM proposed model.

SFOR-ELM applied data mining techniques to find the most features that affected the sales of clothes and these features have been selected as the input features of SFOR-ELM model. Then the model has been evaluated using MAPE, RMSE, and R2.

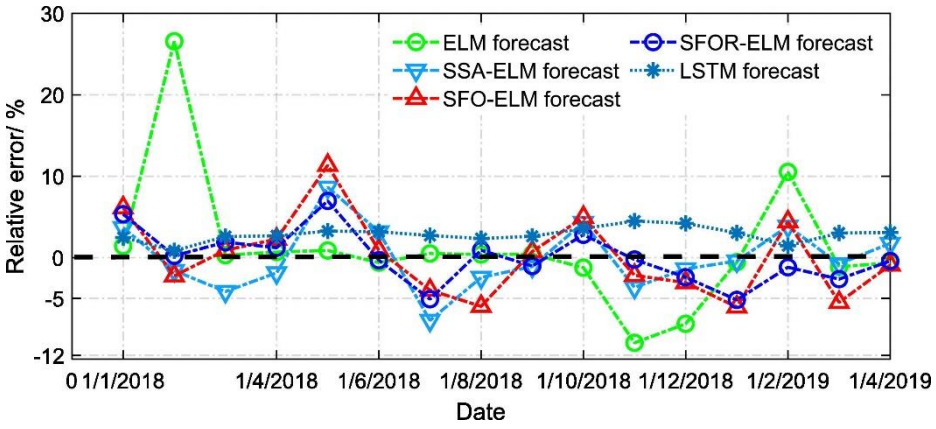


Figure 10: SFOR-ELM proposed model error rate.

Figure 10 above describes the error rate of all these models and SFOR-ELM ranged between 5 and -5% which was the best one out of all the models. However, the LSTM forecast have been close to it in most cases SFOR-ELM was barely better than LSTM. Although of the good results of SFOR-ELM as it was competitive results, there is limitations to solve it should improve the solving ability, the generalization ability and its scalability of online sales forecast the approach need to be improved [20].

3. Evaluating the performance of deep neural networks and other data mining techniques (DNN) to perform predictions in fashion retail stores. In figure 11 below is their proposed model and their steps to build the model. After preprocessing the data, the data set was split into two data sets according to the season. To make sure the items will not overlap with each other if the items in two different seasons. To improve the model results 3 steps were done. Apply greedy algorithm to it to find the predictions factors of the data to integrate into the model, optimize process of the parameters then a testing is done to the model. The model is trained with historical data and tested with future items.

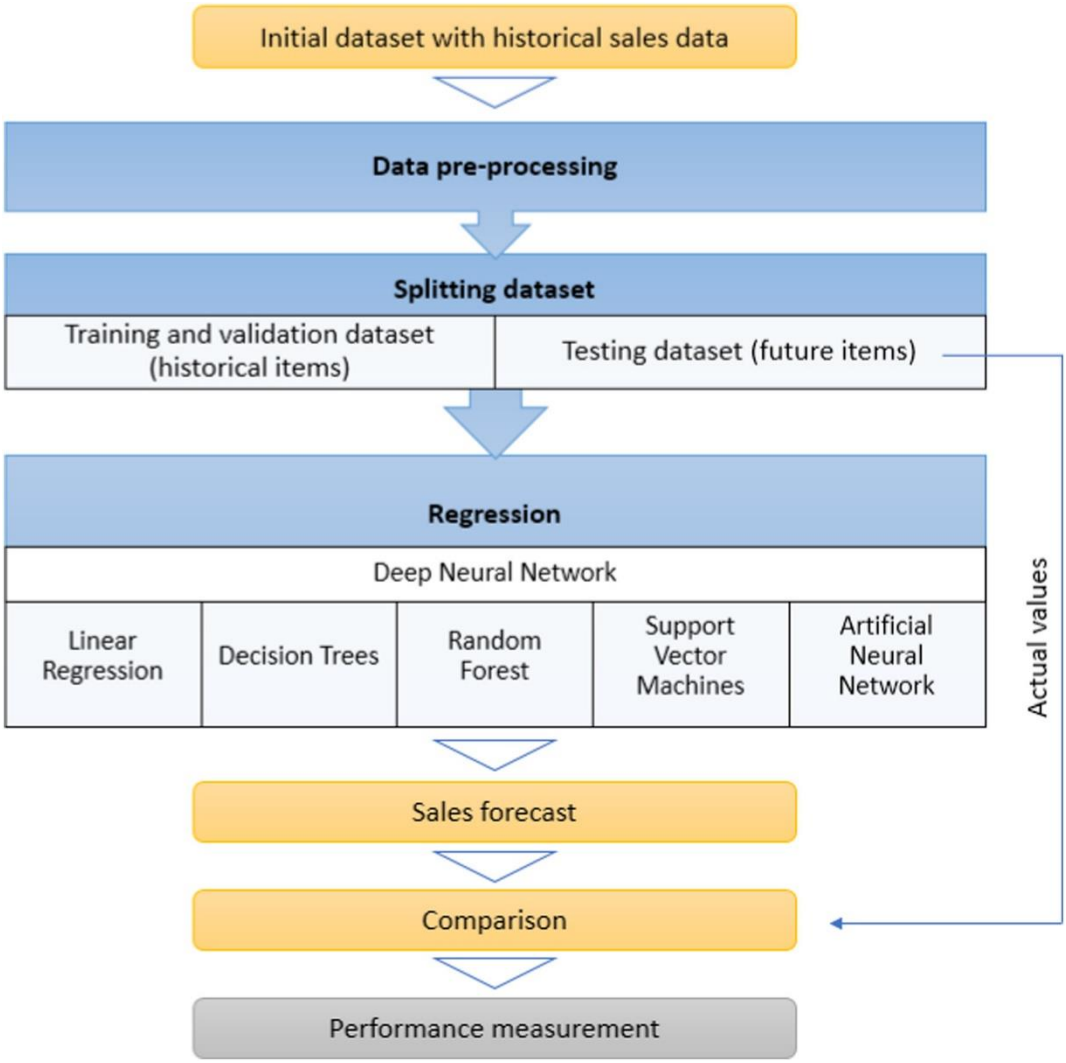


Figure 11: deep learning proposed model

This step was to optimally determine the variables and the parameters to good an improved model performance. The first training dataset was divided into k folds by using k-fold cross validation where k is equal to 10. The best parameters were found by

repeating the training multiple times until the best parameters were found. To select the best predictive variables combinations for the model to consider in training and testing. Greedy feature selection technique was used at each stage to try and find the best optimally local choice. dependent variable was associated with each independent variable, as the model looks for one independent variable in 10 total possibilities. At every possibility the parameters were hyper optimized that used a grid brute force search of the defined values for each parameter under analysis. The model performance was evaluated using root mean square error (RMSE), mean absolute percentage error (MAPE), mean absolute error (MAE). The performance was determined by finding the mean value after 10 iterations. The used algorithms are Depp Nerul Networks (DNN), Random Forest (RF), Logistic regression (LR), support vector regression (SVR), and Decision trees (DT). After applying the greedy feature selection one feature were selected as the most important product variable as seen in figure 14 below.

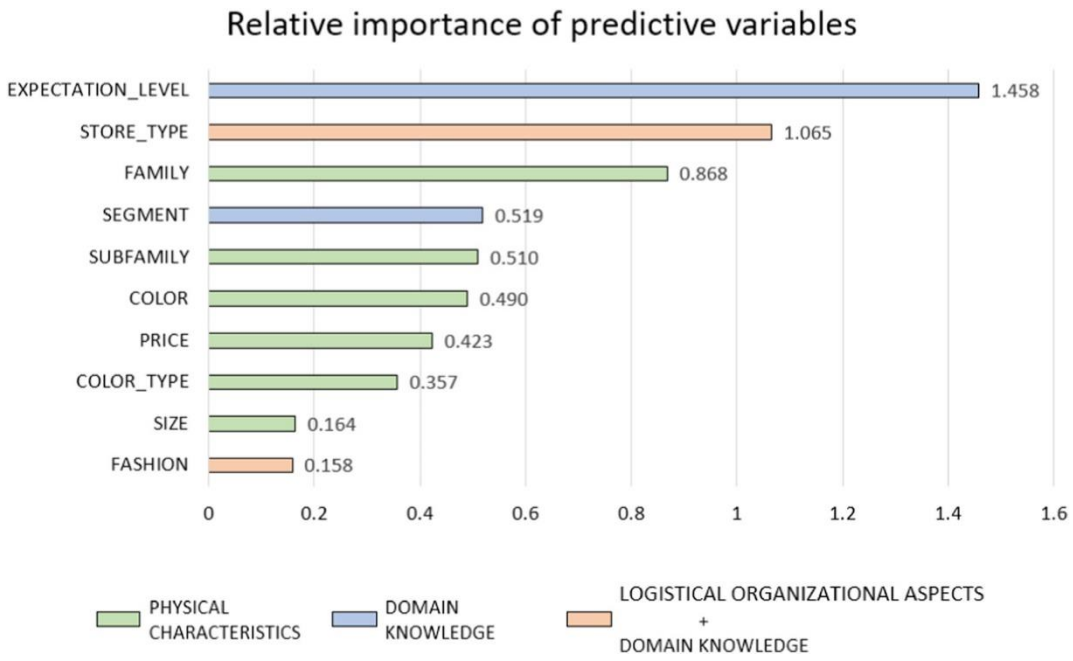


Figure 12: most important feature

Figure 13 shows us the performance of the model with the expectation level feature and figure 14 shows us the performance without this feature.

		Regression technique					
		DNN	DT	RF	SVR	ANN	LR
Evaluation metric	R^2	0.875	0.812	0.796	**0.923**	0.807	0.612
	RMSE	820.1	937.4	983.9	**618.5**	1045.9	1696.1
	MAPE	0.147	0.117	0.141	**0.069**	0.208	0.387
	MAE	494.3	434.8	470.7	**253.9**	663.1	1029.2
	MSE	689,494	918,190	1,010,160	**393,851**	1,125,462	2,911,026

Figure 13 performance of the model with the expectation level feature

		Regression technique					
		DNN	DT	RF	SVR	ANN	LR
Evaluation metric	R^2	0.633	0.412	0.423	0.560	0.461	0.405
	RMSE	2265.1	2684.3	2685.2	2453.2	2477.9	3115.5
	MAPE	0.419	0.454	0.444	0.435	0.449	0.472
	MAE	1397.6	1618.1	1611.2	1566.0	1537.8	1756.8
	MSE	5,130,620	7,205,361	7,210,423	6,018,366	6,140,134	9,706,410

Figure 14 performance of the model without the expectation level feature

it was clear the impertinence of finding the most important feature to work on while training and testing the model as it drastically improved the model performance on all the test measure. The SVR was the best performing algorithm and the closest one to it was DNN that was after optimizing the parameters to improve the model efficiency. But in figure 16 the DNN outperformed all other models, showing us how strong deep learning is when there is lack of historical data and how efficient it is in sales forecasting field [21].

4. This paper proposed a demand forecasting model for an e-commerce website that is shipping worldwide. The proposed model at first will start with a two-step clustering algorithm [22] that is an improved algorithm based on BIRCH [23], then C-XGBoost model will be applied for each cluster of the resulting clusters with XGBoost algorithm, that it is proved it is an efficient algorithm in predicting after being used with many data analysis. Then to improve the model performance an A-XGBoost model is used to show the power of ARIMA and XGBoost model, hence at the end a C-A-XGBoost model is

applied, which puts in considerations factors that is affecting the sales of items. Figure 15 shows us the structure of the algorithm.

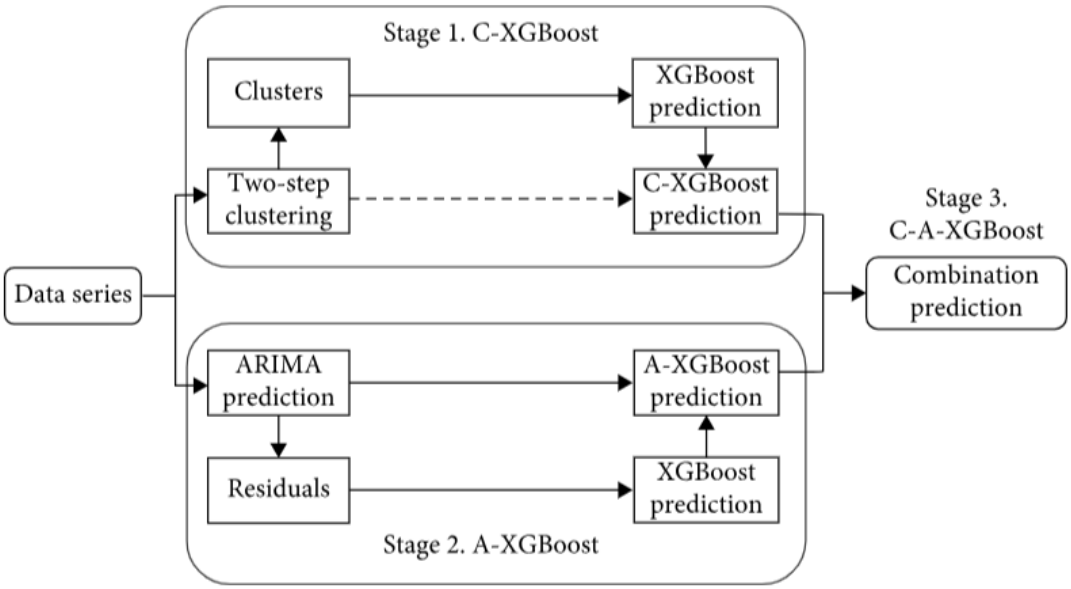


Figure 15: proposed C-A-XGBoost model

The prediction from C-XGBoost and A-XGBoost are combined to give us the final predictions of the model. The results compared ARIMA, XGBoost, C-XGBoost, and A-C-XGBoost with the actual sales value. The results have proven that the proposed model A-C-XGBoost was the best fitting performance out of all the other models as it gave the closes predictions to the actual sales of the products. As you can see in figure 16 below.

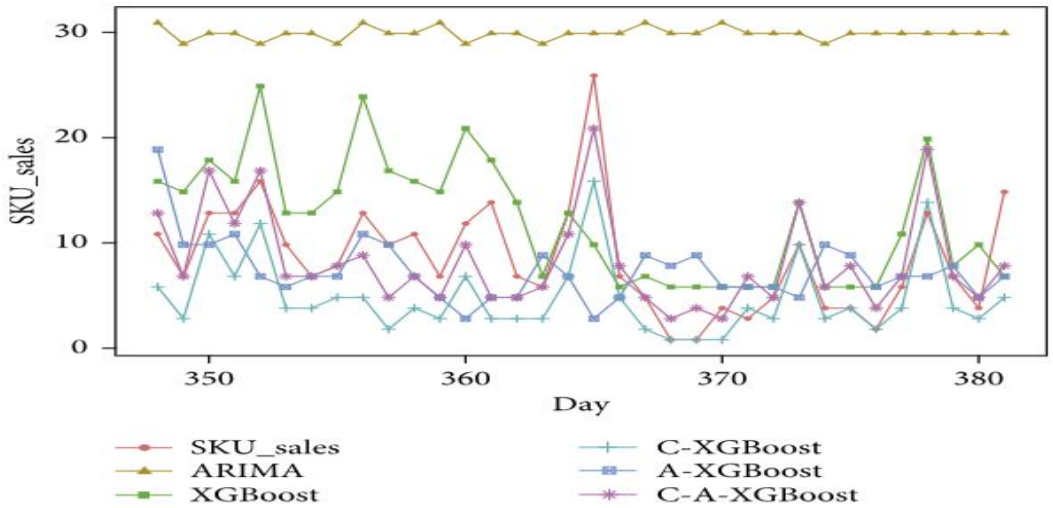


Figure 16: A-C-XGBoost model score

The C-XGBoost is a combination between clustering and XGBoost to find sales of future items in future predictions. Two-step clustering algorithm divides data into different clusters based on specified features, then it will be used as an important decision in the model final

predictions. A-XGBoost is proposed to take advantage of the ARIMA model as it can predict the future direction of the data. XGBoost is used to overcome ARIMA Disadvantages because it can deal with non-linear data. Arima predicts the linear part of the data set and XGBoost will predict the non-linear part of the data, the input and output are switched between ARIMA model and XGBoost model to get the full predictions of both linear and non-linear data [24].

5. This paper proposed an LSTM-XGBoost model to forecast demand in e-commerce stores. They combined the two models based on weighted set the model was trained on the last 2 years company data. Their proposed LSTM-XGBoost is better than a single time series forecasting model. You can see the structure of the two models in figure 17 below.

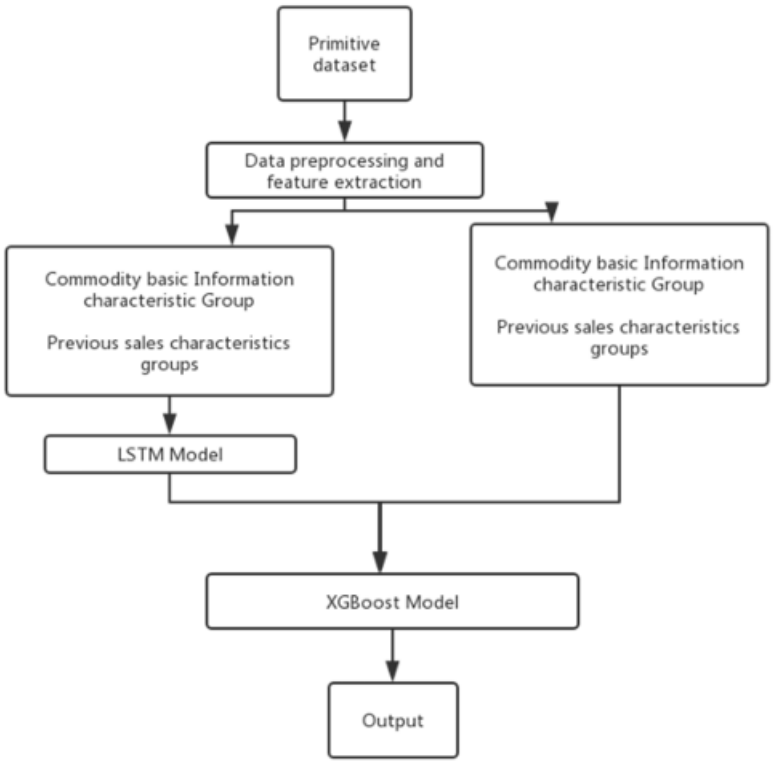


Figure 17: proposed XGBoost-LSTM Model

The model was trained with data from 2017 to 2019 78 groups out of 1096 sales time series was used in building the dataset. The result of the proposed XGBoost-LSTM Model was much higher than the original XGBoost model as it gets the most advantages out of the two algorithms which added a lot of value for the company that used that model. And LSTM parameters can be adjusted to find the most accurate prediction model [25].

2.3 Analysis of the Related Work

Reference	Algorithm	Domain
https://doi.org/10.1016/j.epsr.2019.106073	DRNN-GRU	Residential building electricity demand
https://doi.org/10.1016/j.procir.2021.03.081	LSTM, BLSTM (Bi-directional LSTM)	Time series forecasting in supply chain management
https://doi.org/10.1016/j.cie.2020.106796	Cross-temporal hierarchical framework with LSTM	Supply chain forecasting
https://doi.org/10.1016/j.cie.2020.106435%20	LSTM	Demand forecasting
https://doi.org/10.1080/16258312.2021.1967081	ARIMA, Holt model	Forecasting in pharmaceutical industry
https://doi.org/10.1016/j.cie.2022.108935	SFOR-EMR, SFOR, EMR	Sales forecasting in e-commerce websites
https://doi.org/10.1016/j.dss.2018.08.010	DNN, RF, LR ANN, DT, SVR	Forecasting in fashion retail
https://doi.org/10.1155/2019/8503252	ARIMA, XGBoost, clustering, two-step clustering	Forecasting demand in worldwide e-commerce
10.1088/1742-6596/1754/1/012191	XGBoost-LSTM	Sales Forecasting in e-commerce

In the papers I have collected they aim was to build a sales forecasting model and they have used various machine leering algorithms and techniques like LSTM, SVR, ARIMA, HOLT and XGBoost. Seeing their papers I have found that in order to get the best performance out of your model it is better if your model is hybrid meaning it should have two algorithms each one should train subset of the dataset, as in many cases the data set can not be trained by a

single model like the case of ARIMA and XGBoost with their proposed model A-XGBoost ARIMA trained linear data and XGBoost trained non-linear data. So, to get the best model performance and accuracy my work should contain a hybrid model.

I have noticed the important of optimizing parameters and choosing the most important features from the dataset that can affect the outcome of the model. In the training process of the model, it can drastically improve the outcome of your model if it was done correctly.

My paper is discussing building a sales forecasting model that it will have a better accuracy compared to related state of the art, and my model will be deployed into a website so the business owners can effectively use the tool that I'm providing.

3 Proposed solution

3.1 Solution Methodology

Proposed model for this study will focus on trying various machine learning techniques and deep learning techniques to find the most accurate and the best model to use in building a sales forecasting model for retailers.

Machine learning techniques like decision tress, random forest, linear SVM

Time series models and Neural networks such as ARIMA and LSTM

Experimental setup

The model will be trained on historical sales data for 45 Walmart stores that are in different regions.

The model will be trained and evaluated with a Walmart dataset that is divided into 2 csv files.

The first file is train.csv which includes the following features.

- Store - the store number
- Dept - the department number
- Date - the week
- Weekly_Sales - sales for the given department in the given store
- IsHoliday - whether the week is a special holiday week

The second file is features.csv which includes the following features.

- Store - the store number
- Date - the week
- Temperature - average temperature in the region
- Fuel_Price - cost of fuel in the region
- MarkDown1-5 - anonymized data related to promotional markdowns that Walmart is running. MarkDown data is only available after Nov 2011, and is not available for all stores all the time. Any missing value is marked with a NA.
- CPI - the consumer price index
- Unemployment - the unemployment rate
- IsHoliday - whether the week is a special holiday week

The data does not include all of the holidays it only has:

Super Bowl: 12-Feb-10, 11-Feb-11, 10-Feb-12, 8-Feb-13
Labor Day: 10-Sep-10, 9-Sep-11, 7-Sep-12, 6-Sep-13
Thanksgiving: 26-Nov-10, 25-Nov-11, 23-Nov-12, 29-Nov-13
Christmas: 31-Dec-10, 30-Dec-11, 28-Dec-12, 27-Dec-13

The model will predict the test.csv file which only has all of the features that in train.csv file but the model will predict the weekly sales.

Model evaluation
The model will be evaluated using root mean square error (RMSE), mean absolute percentage error (MAPE), mean absolute error (MAE).

3.2 Functional/ Non-functional Requirements

Functional Requirements

1. User Authentication and Authorization:
    - The system will include user authentication and authorization to ensure that users data will be secured always.
    - System should have different roles and every one access should change according to their role.
2. Simple user interface
    - The user interface should be good looking and easy to understand so users can easily get used to the interface and know how to deal with it optimally.
    - the user interface should be implemented using new and modern libraries to ensure that it will be optimal and that it looks good.
3. Data integration and processing
    - The system will be built using node, express to optimize the back end of the application and the database will be built using mongo database to safely save the required data in the database and have access to it.
    - The system will have a restful APIs to manage communications between frontend and backend.
    - The data must be exchanged properly from the database and in the database.
4. Sales forecasting
    - Utilizing powerful machine learning and deep learning algorithms to build the model.
    - The system should give users options for the forecasts as it can be short term forecasts and long-term forecasts.

5. Data Visualizations

    - The business data of your company should be presented to the user in a simple way while making sure to display important data to the user.
    - It should be easy to customize what data will be displayed for the user and it should be easy to change how the data is presented to the user.

Non-functional requirements

1. Performance

    - The system should be responsive and must provide forecasts fast for the user.
    - Frontend and backend communications should be handled quickly as the user should not wait a lot of time for that type of interactions.

2. Maintainability

    - The code should be easy to read and to understand.
    - It should be easy to add updates to the website to add features to the system and to easily fix bugs if it was found.

3. Security

    - Data transfer between database should be encrypted to hide users' data.
    - Authorization and authentication in the login function of the system to keep users' personal data safe.

4. Usability

    - Develop user with documentations to easily understand what the functions does in the system and to know how to use it.

5. Compatibility

    - The system should support wide web browsers support to ensure that a lot of users will use it.

3.3 Design and Simulation

❖ Package Diagram

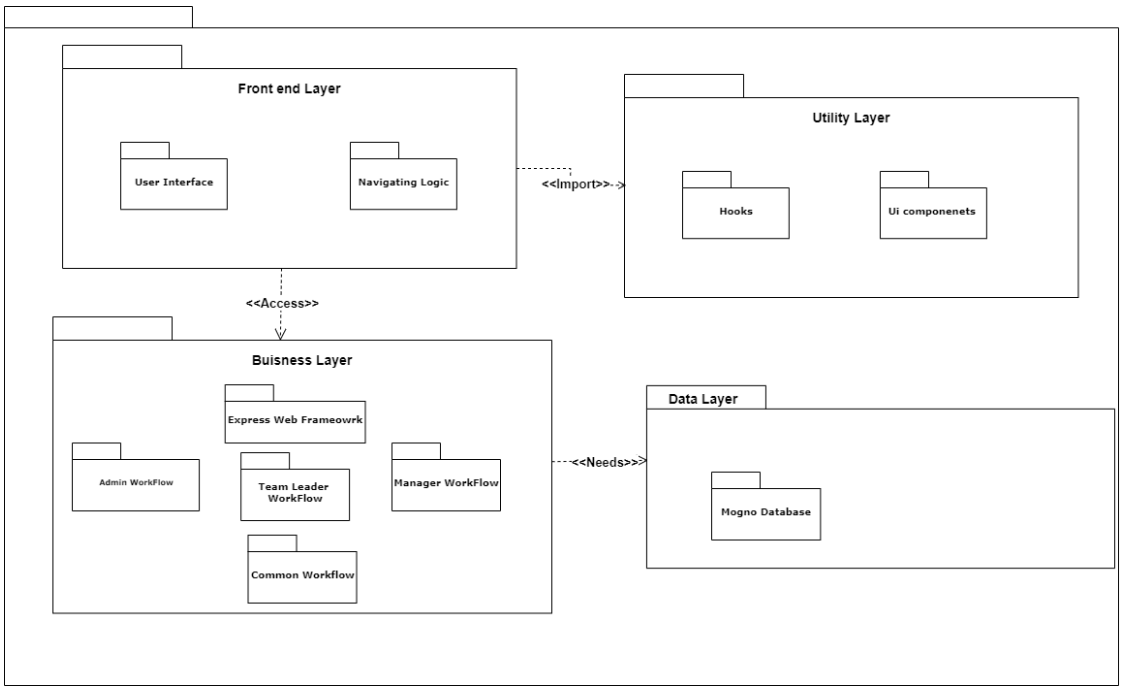


Figure 18 Package diagram

The implementation of the System will be in 4 packages

- **Front End layer: User Interface and navigation logic**
- **Utility Layer: hooks and components**
- **Business Layer: controller and router**
- **Data Layer: Data Models**

❖ **Use case diagram**

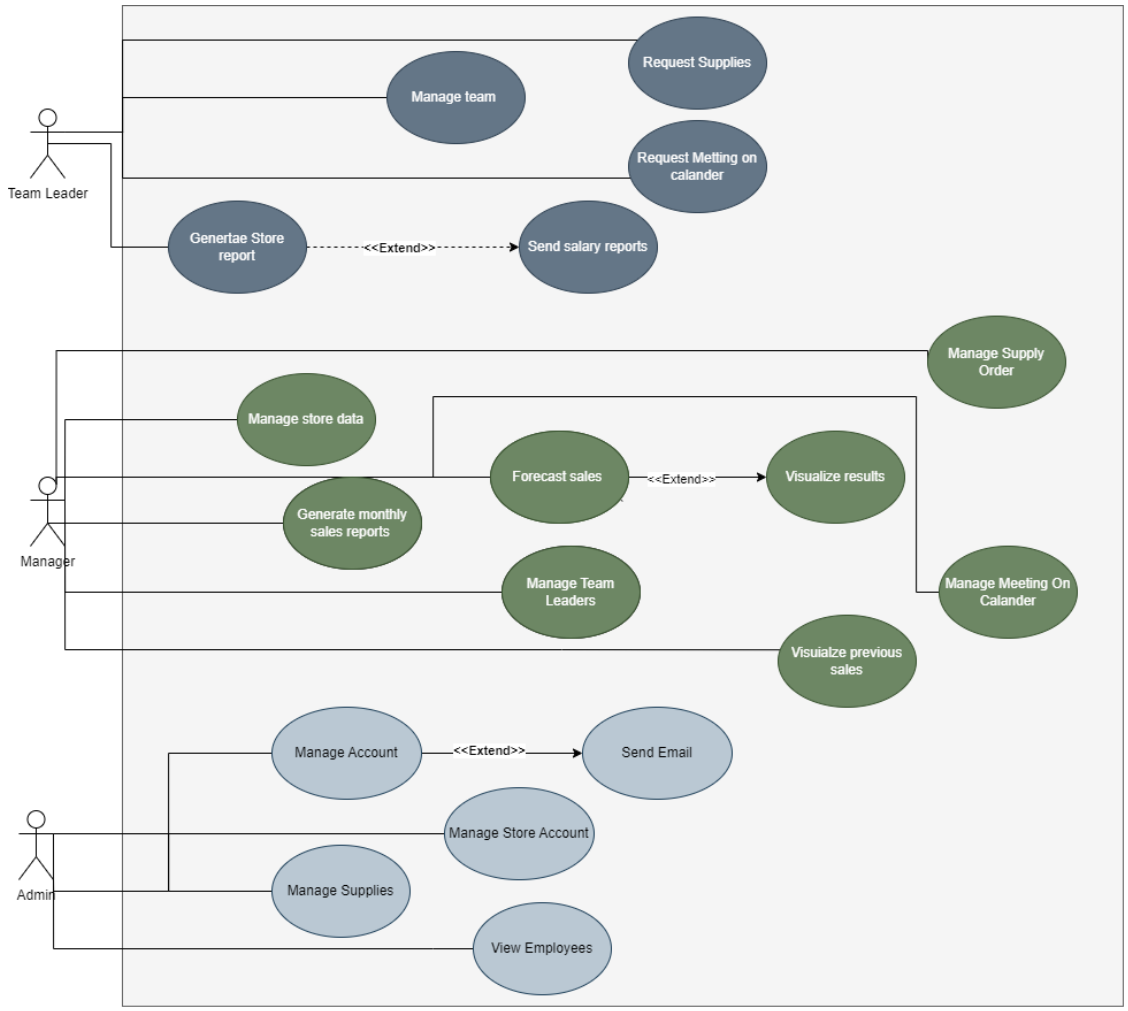
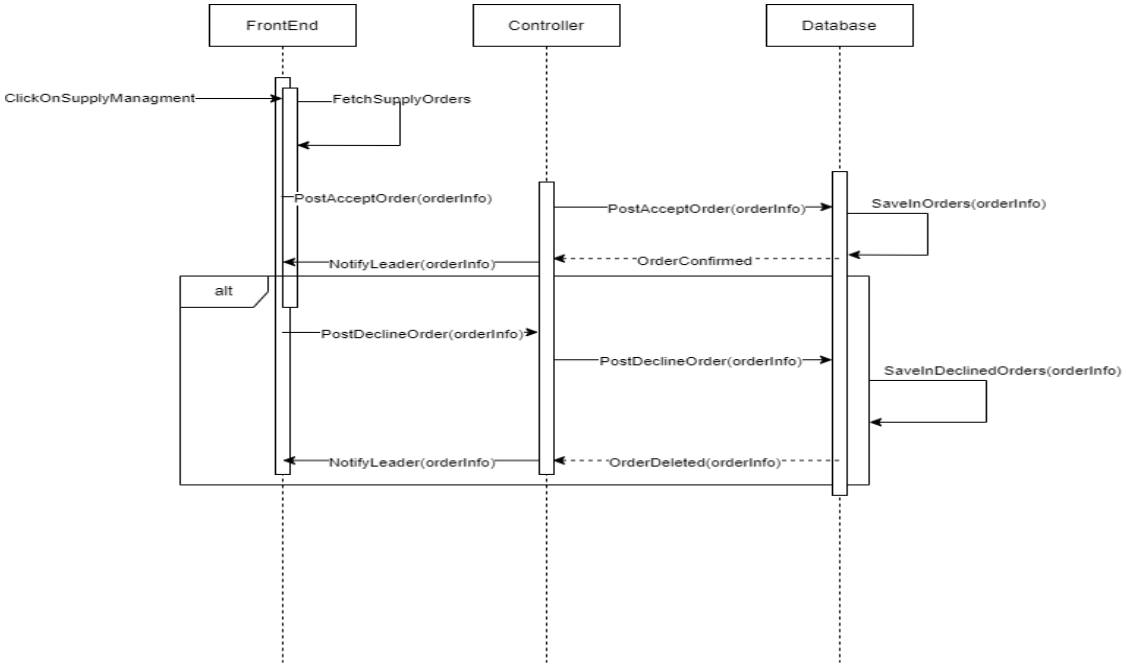
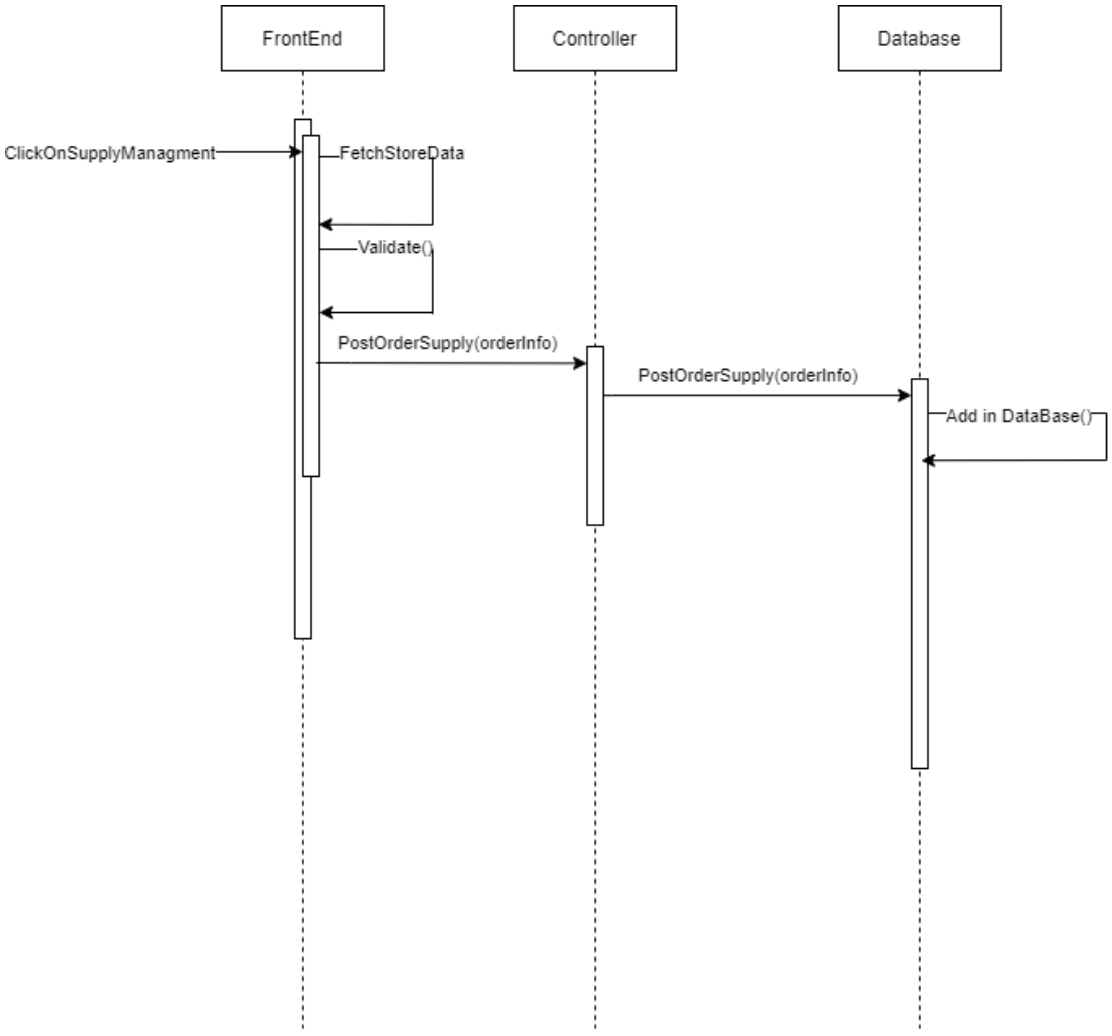
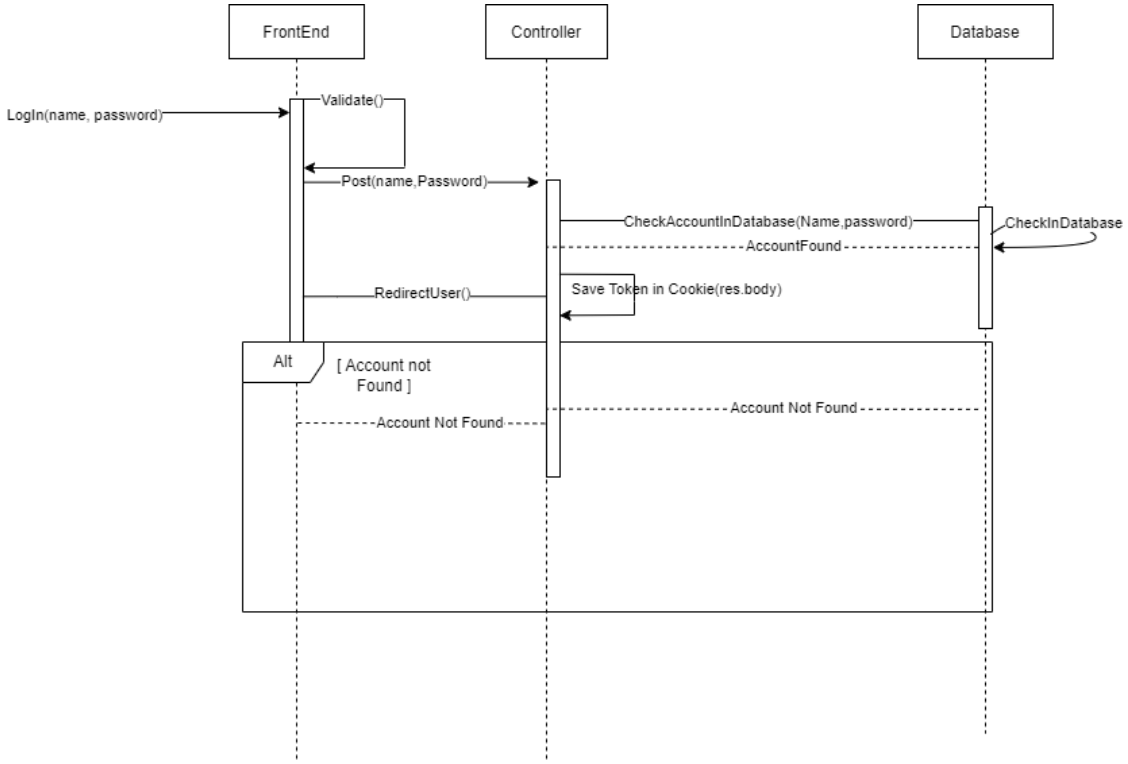


Figure 19 use case diagram

Sequence diagrams







Figures 20, 21 and 22 System sequence diagram

4 Conclusions and Changes

1. Model Architecture: The model is a sequential neural network composed of several layers, primarily LSTM (Long Short-Term Memory) and Dense layers. Here's a breakdown of the architecture:

- **Input Layer**: The input layer accepts data with a batch shape of (None, 10, 12), implying a sequence length of 10 with each sequence having 12 features.
- **LSTM Layers**: There are three LSTM layers sequentially stacked.
    - The first LSTM layer has 128 units with a return sequence of True.
    - The second LSTM layer has 64 units with a return sequence of True.
- **Dense Layers**: Following the LSTM layers, there are two Dense layers.
    - The final Dense layer has a single unit with linear activation, suitable for regression tasks.

2. Optimizer Configuration: The model utilizes the Adam optimizer with a learning rate of 0.01, beta_1 of 0.9, beta_2 of 0.999, and epsilon of 1e-07. It doesn't employ weight decay, gradient clipping, or AMSGrad.

3. Experiment Results: The model's performance metrics on a validation or test dataset are as follows:

- Root Mean Squared Error (RMSE): 0.136
- Mean Absolute Error (MAE): 0.099
- Mean Squared Error (MSE): 0.0186
- R-squared: 0.785

4. Performance Analysis:

- **RMSE and MAE**: Both RMSE and MAE provide insights into the average prediction error of the model. With RMSE at 0.136 and MAE at 0.099, it indicates that, on average, the model's predictions deviate by around 0.136 and 0.099 units from the actual values, respectively.
- **MSE**: MSE quantifies the average squared difference between predicted and actual values. A lower MSE indicates better model performance in terms of accuracy.
- **R-squared**: R-squared measures the goodness-of-fit of the model.

5 Implementation

Website Implementation

The website is built with MERN Stack which is a pre-built stack based on JavaScript technologies which stands for MongoDB, React, Node and express

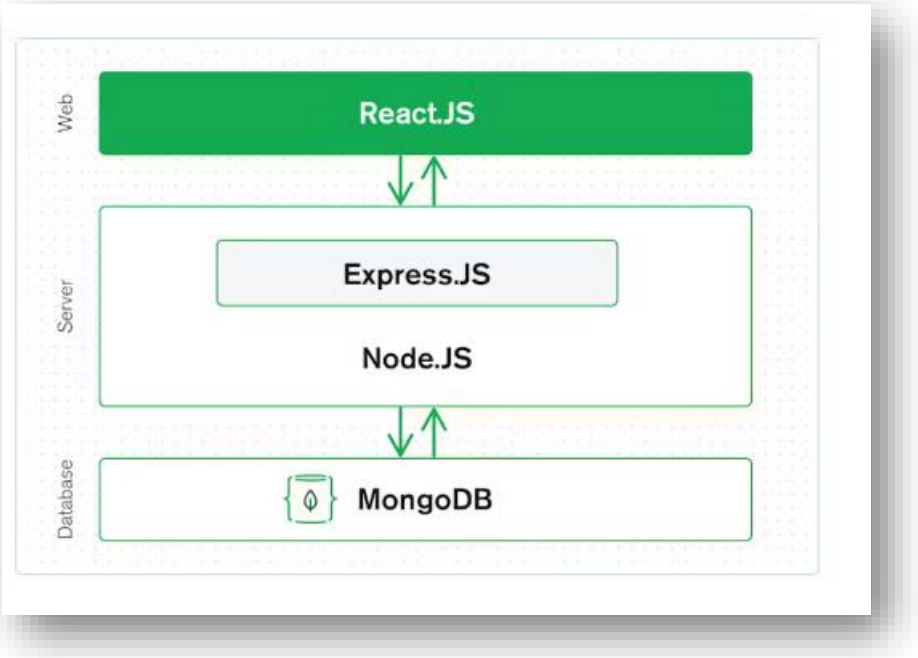
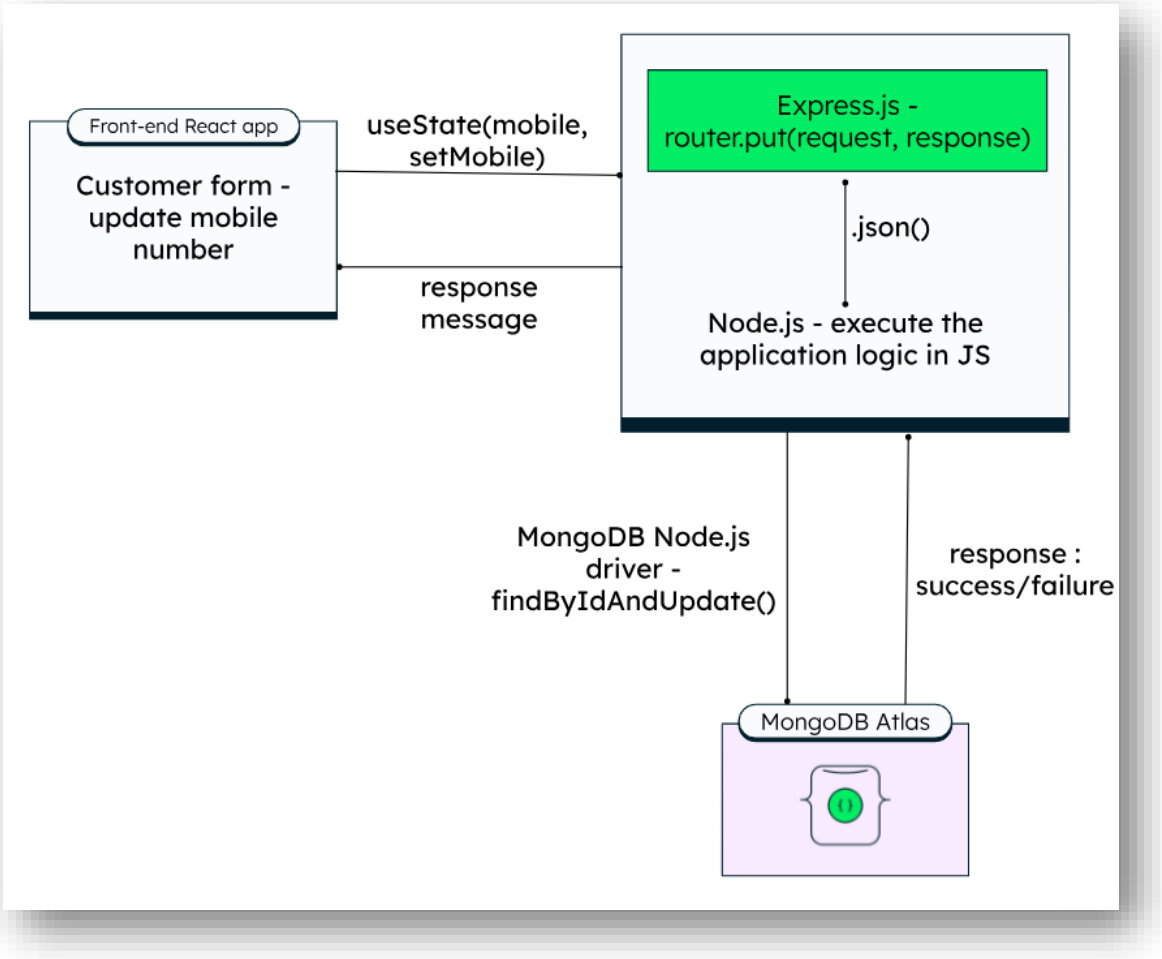


Figure 23 mern stack architecture

- React.js (Web Tier): React.js is a JavaScript framework used in front end software development, one of its features is it allows us to make complex and good-looking modern websites design using simple components that helps in reusability, and flexibility. React components are easily connected to the Database and the back end, then these components are rendered to display data in it (Mern)

- Express.js and Node.js (Server Tier) : Express.js is a server-side framework running inside of Node.js (Mern)Node.js is the run time environment used to build scalable network applications (Node)that is able to handle upcoming requests GETs or POSTs, using MongoDB Node.js drives to get data from the back end and allows us to do any required operations on the data

- MongoDB database tier: non-SQL database used in MERN stack to handle data



in the website Example of simple request/response with MERN stack

Figure 24 example of request

Example to allow user to update his mobile number which is sent to the router that make a put request that have the request and the response then the request is sent to the database which will make query to find User by Id and update the user with the id and the response is saved and returned to the user in case of an error the error is returned if the response with a success the mobile number is set with the useState to set the Mobile value using the function of setMobile

❖ **Application structure**

The application is divided into two parts one part for the front-end and the other is for the back end

- Front-end consists of all the pages and the components that is used in the pages they are reusable components that are easy to edit and export to use in another react page, the hooks is in the front-end components which are used to fetch information's from the database that are used multiple times in multiple pages and components of the page,

Components

Reusable components: Duplicate Ui elements such as header nav bar and side bar and styles of the form, dialog components that can be used with different pages of the page, reusable and flexible pop up forms that only displays the field so any addition to the database can be changed and can be used with most of the features, data grid tables same as the flexible forms can be used with anything on the table they are just the styles of the table

Dialog pop up: pop up dialogs can be used in two cases first case to delete and the other for confirm both are handled in the same pop up component by making its props takes text to be displayed and the option either delete confirm

```
const Dialog = ({ isOpen, onClose, onConfirm, title, message, actionType }) => {
  if (!isOpen) return null;
```

Figure 25 screen shot of dialog code

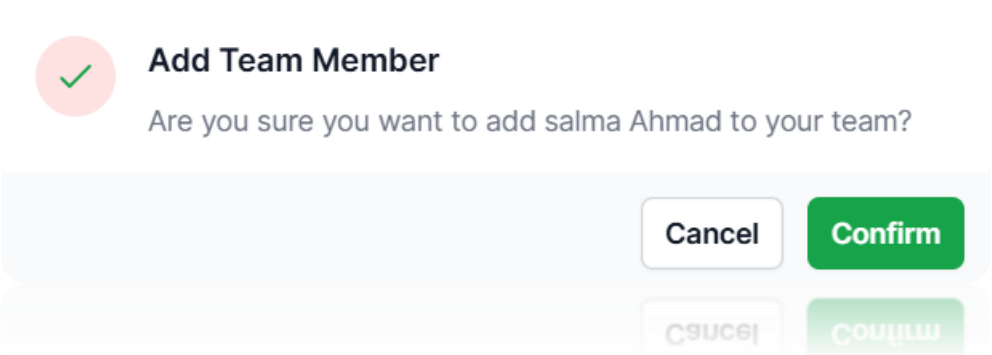


Figure 26 Confirm Dialog

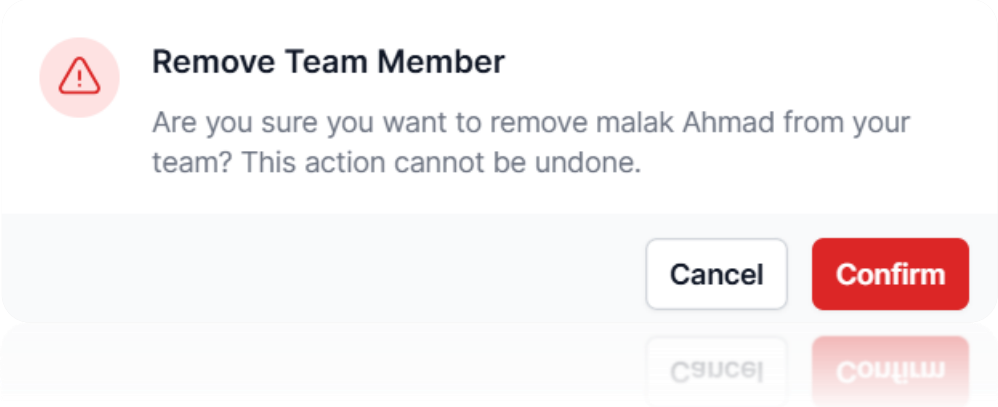


Figure 27 remove dialog

Data Grid Component: Table used display Important data to the user and can be used based on the class it is being used with by giving its props that have an array of fields to be rendered in the table and actions such as add remove or edit

```
const DataGridComponent = ({ columns, actions }) => {
  const theme = useTheme();
  const { staffMembers, isLoading, fetchStaffMembers, token } = useFetchStaffMembers();
  const [selectedMemberId, setSelectedMemberId] = useState(null);
  const [isEditModalOpen, setIsEditModalOpen] = useState(false);
  const [isConfirmationDialogOpen, setIsConfirmationDialogOpen] = useState(false);
```

Figure 28 Data Grid Props

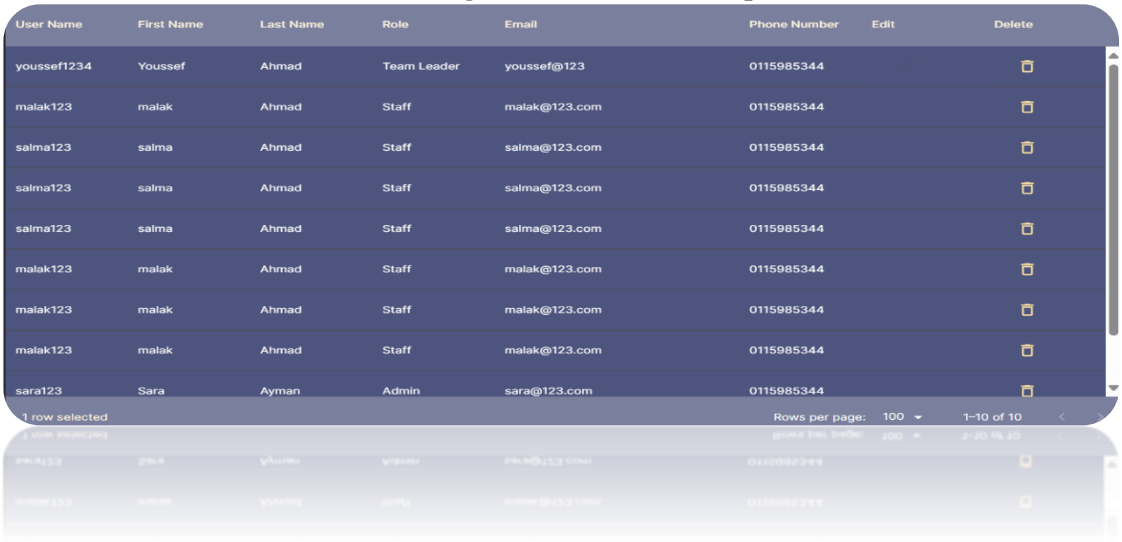


Figure 29 staff Data Grid

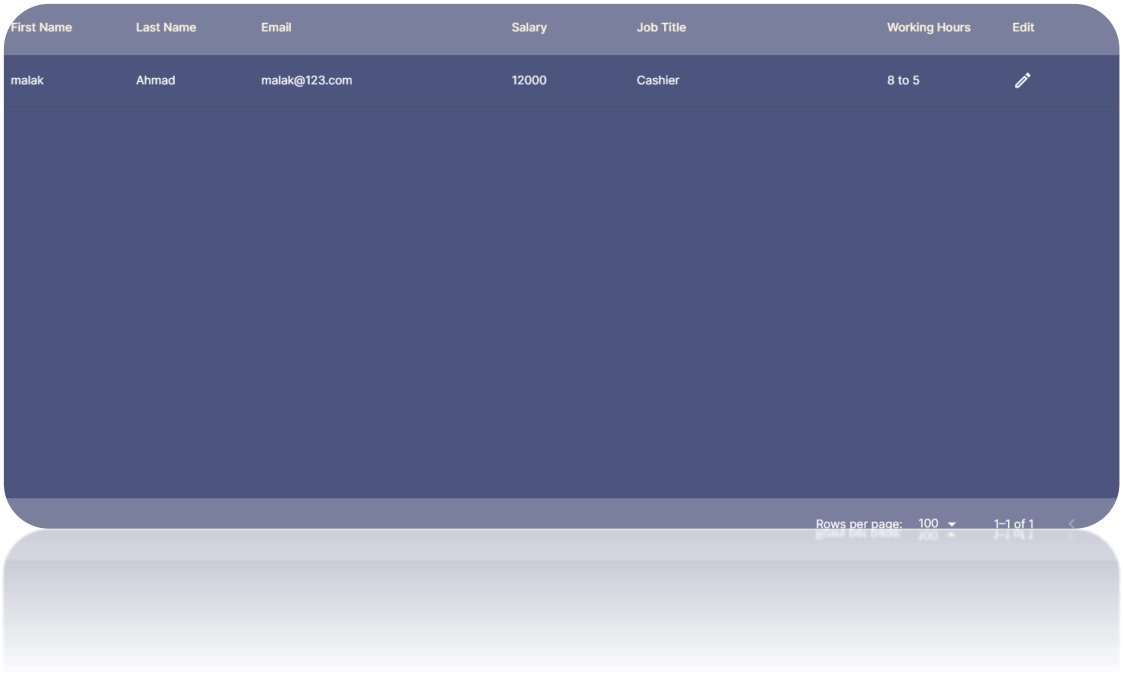


Figure 30 team data grid

Reusable forms and pop-up forms: Same as the dialog and the data grid you can control what object can be rendered in the form by sending a props of an array of fields to the form the form is sent the type of text to get the field responsible to taking that input and it is being sent the field name as well

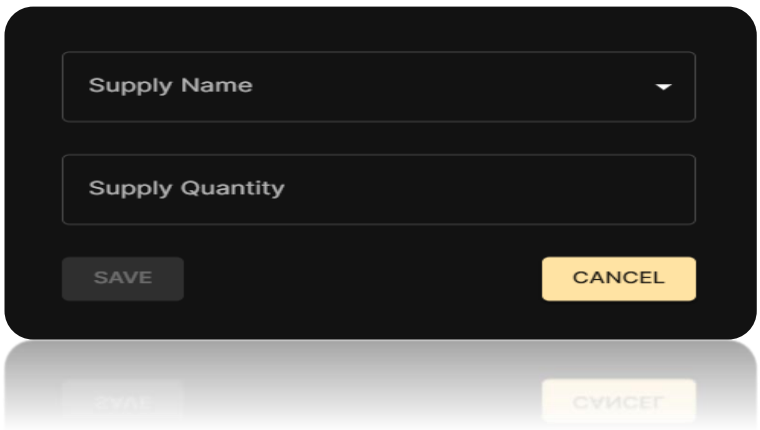
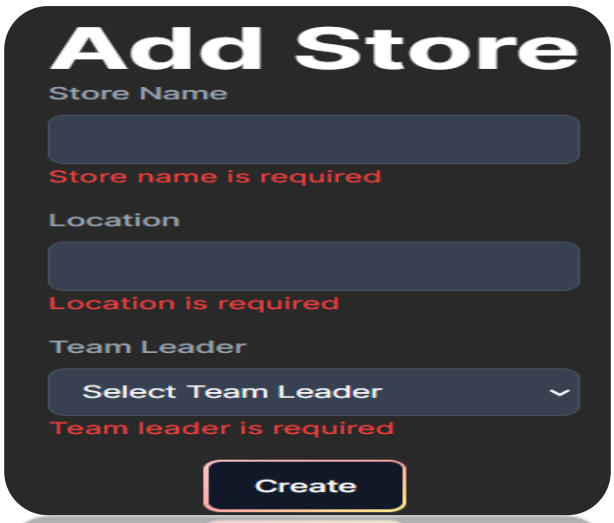


Figure 31 order supply pop up form



Figures 32 & 33 Add supply and add store

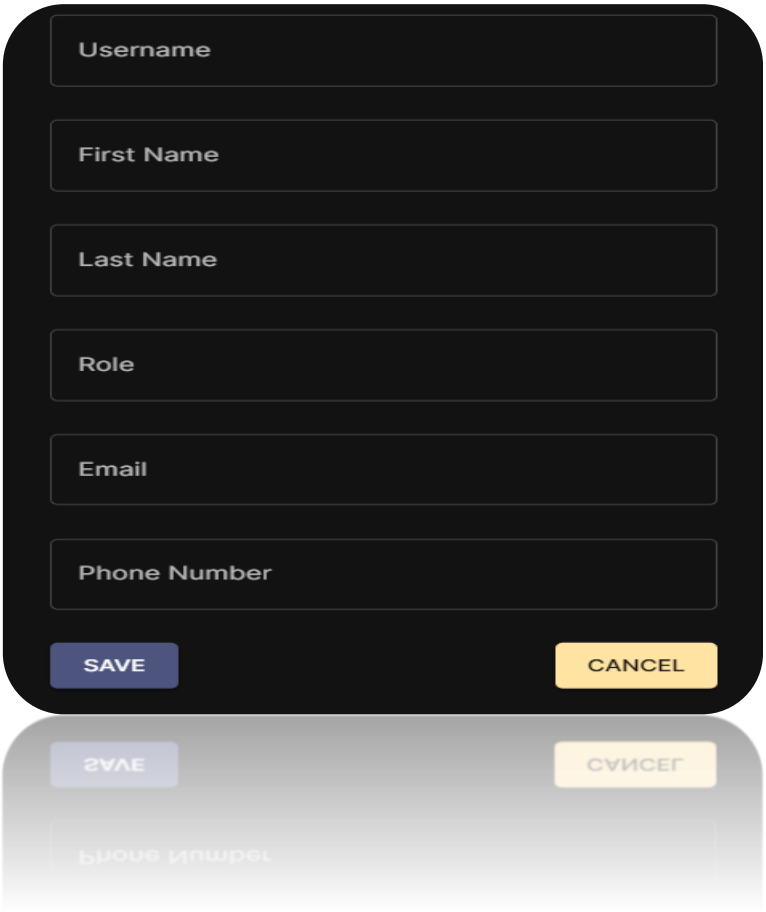


Figure 34 Edit account pop up form

Reusable card: Card used to display team members information for different team members, and they can either add or remove someone to their teem as the cards takes prop of the employee information and the action either add or remove

```
const TeamMemberCard = ({ id, name, email, jobType, onActionClick, actionType, profilePicture }) => {
  const [isModalOpen, setIsModalOpen] = useState(false);
  const theme = useTheme();
```

Figure 35 Card props

Figure 36 Remove card

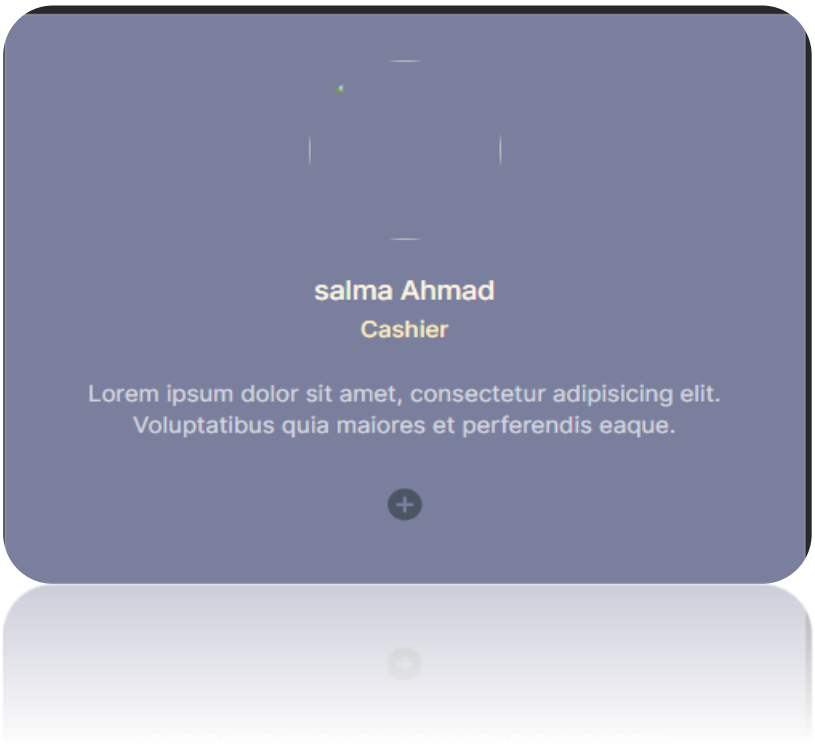


Figure 37 add card

Chats: Important part of any business application is visualization sales and important business data can be used with any important business data as it is reusable in more than one class

Figure 38 bar chart

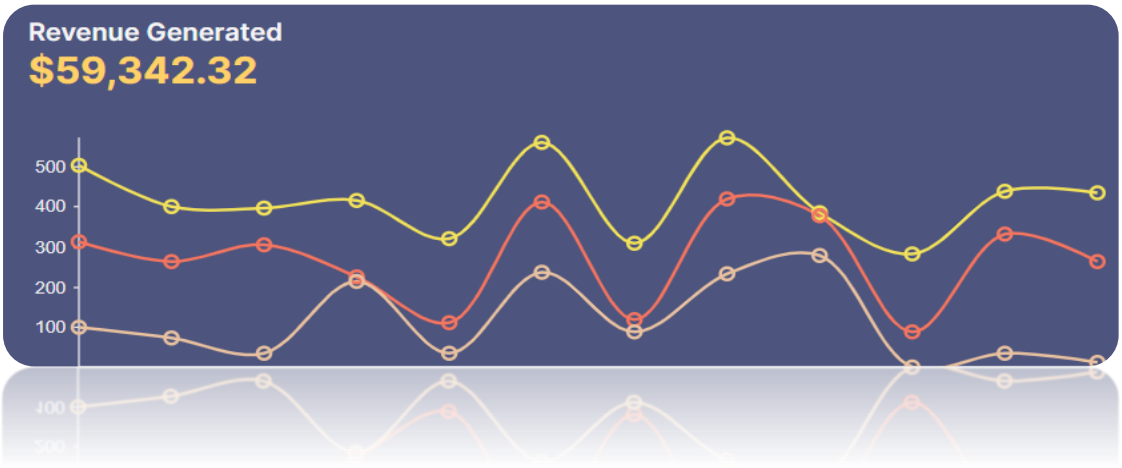


Figure 39 Line chart

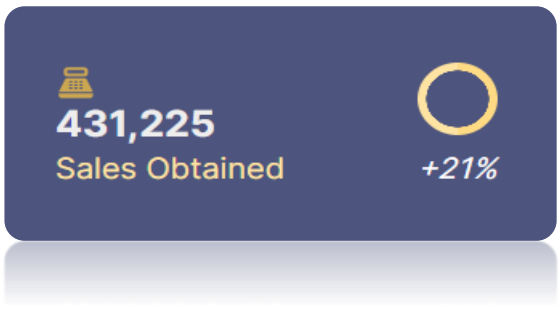


Figure 40 circular chart

Hooks: “Hooks allow us to "hook" into React features such as state and lifecycle methods.” **[w3 schools]**

useEffect Hook: “useEffect is a React Hook that lets you synchronize a component with an external system.” [react documentations]

- used in the application to fetch user token from Api authentication as the token contains important data sued as the user id which is used to make sure only your data is retrieved for integrity of the data and sometimes used to get any other needed data from the database, it contains the user name and the role of the user to make sure he is redirected to his/her page with ease

```
const useAuthToken = () => {
  const [userId, setUserId] = useState(null);
  const [userName, setUserName] = useState(null);
  const [userRole, setUserRole] = useState(null);
  const [token, setToken] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    const tokenFromCookie = Cookies.get("jwt");

    if (tokenFromCookie) {
      try {
        const decodedToken = jwtDecode(tokenFromCookie);
        const { _id, username, role } = decodedToken.UserInf

        setUserId(_id);
        setUserName(username);
        setUserRole(role);
        setToken(tokenFromCookie);
      } catch (error) {
        console.error("Error decoding token:", error);
      }
    } else {
      console.warn("No token found in cookies");
    }

    setIsLoading(false);
  }, []);
    console.log("hook",userName)
  return { token, userId, userName, userRole, isLoading };
};

export default useAuthToken;
```

figure 41: fetch token hook

- **fetch staff member's hook:** used to get all the staff from the database and later the data can be filtered by the page that uses the page to only use relevant data for this page, this hook cannot be used without token (to ensure that the user is logged in and should have access to this data).

```
import { useState, useEffect } from "react";  4.2k (gzipped: 1.8k
import axios from "axios";  57.5k (gzipped: 21.1k)
import Cookies from "js-cookie";  1.7k (gzipped: 845)
import {jwtDecode} from "jwt-decode";  1.2k (gzipped: 617)

const useFetchStaffMembers = () => {
  const [staffMembers, setStaffMembers] = useState([]);
  const [userId, setUserId] = useState("");
  const [token, setToken] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    const tokenFromCookie = Cookies.get("jwt");
    if (tokenFromCookie) {
      try {
        const decodedToken = jwtDecode(tokenFromCookie);
        const { _id } = decodedToken.UserInfo;
        setUserId(_id);
        setToken(tokenFromCookie);
      } catch (error) {
        console.error("Error decoding token:", error);
      }
    } else {
      console.warn("No token found in cookies");
    }
  }, []);

  useEffect(() => {
    if (userId && token) {
      fetchStaffMembers(token);
    }
  }, [userId, token]);
```

Figure 42: fetch staff hook part 1

```
const fetchStaffMembers = (token) => {
  setIsLoading(true);
  axios
    .get("/user/users", {
      headers: {
        Authorization: `Bearer ${token}`,
      },
    })
    .then((response) => {
      setStaffMembers(response.data);
    })
    .catch((error) => {
      console.error("Error fetching staff members:", error);
    })
    .finally(() => {
      setIsLoading(false);
    });
};

return { staffMembers, isLoading, fetchStaffMembers, token };
};

export default useFetchStaffMembers;
```

Figure 43 fetch staff part 2

Pages: pages in react renders components and forms to validate and button handles that make requests to the back end using the express router, data handling for forms and uploading image basically all the calls to the back end is made from a react page and sent by the router to its appropriate controller in the back end

Back-end components: the back end of the project is structured into 4 main files router, model and controller and index file (Main file) all use express and node.js all JavaScript frameworks

- **model file:** it is equivalent to java classes including all of its attributes and Mongo DB Scheme to be able to add the attributes to the data base the model is being used by the controller to handle managing data however required
- **Controller file:** have all the controls over a model and can be used to do anything with the model either adding, editing, and deleting an object that is in the database. All the business logic handling is being done in the controller. The controller file must have access to the model file

```
export async function addSupply(req, res) {
  const { supplyName, supplyQuantity, supplyImportance } = req.body;
```

```
export async function getAllSupplies(req, res) {
  try {
    const supplies = await Supply.find();
    res.status(200).json(supplies);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

export async function createOrder(req, res) {
  const {
    storeName,
    orderQuantity,
    orderedSupplyName,
    teamLeaderUsername,
    date,
  } = req.body;
```

```
export async function removeTeamLeader(req, res) {
  const { id } = req.params;
```

```
export const updateAccountData = async (req, res) => {
  const { id } = req.params;
  const { email, phoneNumber, jobTitle, role } = req.body;

  try {
    // Find the user by id
```

Figures 44 & 45 & 46 & 47 example of crud operations such as add and retrieve , edit and remove

- **Router files:** most important part of the MERN stack is it handles all event routing front end events to the back end and makes sure the need controller is the one being called; it makes sure your requests in exchanged between the front end and the back end

```
import express from "express";
import { addEvent, getAllEvents } from "../controllers/eventController.js";
const EventRouter = express.Router();

EventRouter.post("/addEvent", addEvent);
EventRouter.get("/getEvent", getAllEvents);

export default EventRouter;
```

```
import express from "express";
import * as authController from "../controllers/auth.js";
import * as controller from "../controllers/userController.js"
const authRouter = express.Router();

authRouter.route("/login").post( authController.login);

authRouter.route("/refresh").get(authController.refresh);

authRouter.route("/logout").post(authController.logout);

// authRouter.route("/auth").get(login)

authRouter
  .route("/authUserName")
  .post(controller.verifyUser)
  .post((req, res) => res.end());

authRouter.get("/:username", controller.getUser);


export default authRouter;
```

Figures 48 and 49 example of routes

Index.js (Main): like the main class in java but different as the classes is not being made in the main only the router is added to the index file so front end requests can be made to the router and from the router it is passes it to the controller and defining the first part of the router so it can be found in exchanging requests from back end and front end.

The index file has all the project setup as it has the mongo database connection, CORS configuration to allow ports 3000 and 3001 communicate with each other and it starts the back-end server


import cors from "cors"; 5k (gzipped: 2.1k)
import dotenv from "dotenv"; 6.8k (gzipped: 3k)
import helmet from "helmet"; 11.6k (gzipped: 3.1k)
import morgan from "morgan"; 14.2k (gzipped: 5.3k)
import cookieParser from "cookie-parser"; 4.6k (gzipped: 1.8k)
/* Routes */
import clientRoutes from "./routes/clients.js";
import generalRoutes from "./routes/general.js";
import managementRoutes from "./routes/management.js";
import salesRoutes from "./routes/sales.js";
import storeRoute from "./routes/storeRouter.js";
import supplyRouter from "./routes/supplyRoute.js";
import router from "./routes/user.js";
import authRouter from "./routes/auth.js";
import teamRouter from "./routes/teamMembersRoute.js";
import EventRouter from "./routes/eventRoute.js";
/* Configuration */
dotenv.config();
const app = express();
app.use(
cors({
origin: "http://localhost:3000",
credentials: true, // Allow sending cookies from frontend to backend
})
);
app.use(express.json());
app.use(cookieParser());
app.use(helmet());
app.use(helmet.crossOriginResourcePolicy({ policy: "cross-origin" }));
app.use(morgan("tiny"));

```
/* Routes */
app.use("/client", clientRoutes);
app.use("/general", generalRoutes);
app.use("/management", managementRoutes);
app.use("/sales", salesRoutes);
app.use("/user", router);
app.use("/auth", authRouter);
app.use("/team", teamRouter);
app.use("/event", EventRouter);
app.use("/store", storeRoute);
app.use("/supply", supplyRouter)

/* Database */
const PORT = process.env.PORT || 3001;
mongoose
  .connect(process.env.MONGO_URL)
  .then(() => {
    app.listen(PORT, () => console.log(`Server Port: ${PORT}`));

  })
  .catch((error) => console.log(`${error} did not connect`));
```

Figure 50 & 51 index.js file

APIs: Stands for Application Programming Interface, meaning how software with a specific function. Can be defined as contract of service that helps in structuring requests and responses in the development phase to make sure they do its functionality as correct as possible

APIS in project that are made by me a Login authentication API and an API that sends email

Authentication API

1. it is being called every time user logs in the website it takes two variables in the request body username and password and gets from the venv file the secret keys to make tokens
2. Then the username is checked in the back end to make sure that the user exist in the database, and the password is decrypted as it is hashed when the account is created then compare the values to make sure that this is the correct password to that user name
3. Then the token is signed using jsonWebToken library it is signed with the secret key and contains some of the user information such as username id and role and the expiry data shows when is the toke will expire and request the user to log in again

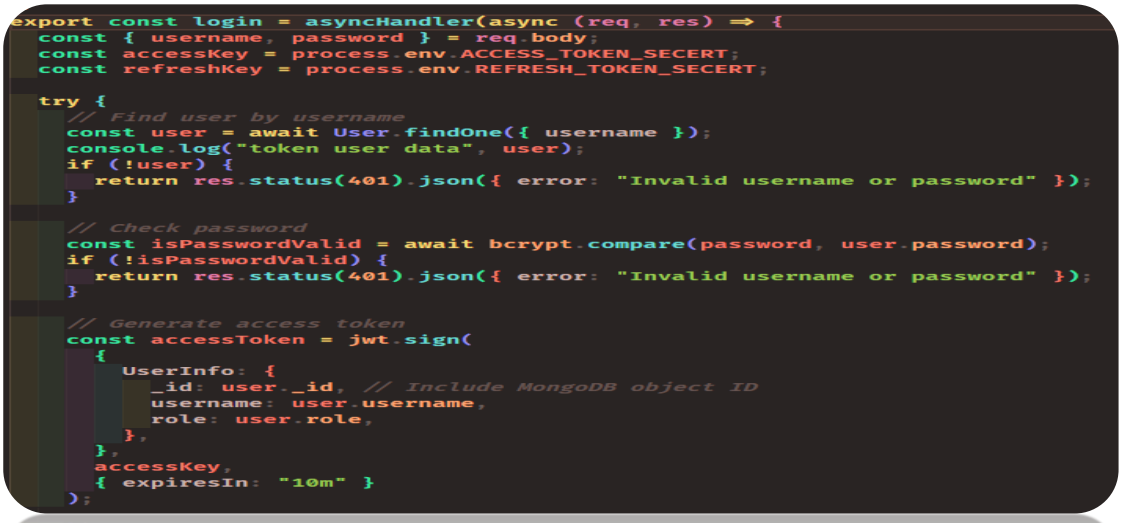
Send Mail Api

1. Starts by making a nodemailer transporter which needs the type of their service and how will you authorize the server the password is created by Gmail Password for this device and must opened by vs code in order for that password to be valid.

2. Then the send mail function which makes the email message structure by including variables such receiver, message and subject and with some styling for the email.

3. The transporter sends that created email then the send email route is being posted and send the email.

4. In order to make the log in Api it had to be done on another server as it was conflicting with the main express server so that's why my application have two backend servers one for the email and other is the main server of the application



```
export const login = asyncHandler(async (req, res) => {
  const { username, password } = req.body;
  const accessKey = process.env.ACCESS_TOKEN_SECERT;
  const refreshKey = process.env.REFRESH_TOKEN_SECERT;

  try {
    // Find user by username
    const user = await User.findOne({ username });
    console.log("token user data", user);
    if (!user) {
      return res.status(401).json({ error: "Invalid username or password" });
    }

    // Check password
    const isPasswordValid = await bcrypt.compare(password, user.password);
    if (!isPasswordValid) {
      return res.status(401).json({ error: "Invalid username or password" });
    }

    // Generate access token
    const accessToken = jwt.sign(
      {
        UserInfo: {
          _id: user._id, // Include MongoDB object ID
          username: user.username,
          role: user.role,
        },
      },
      accessKey,
      { expiresIn: "10m" }
    );
```

```
      },
      refreshKey,
      { expiresIn: "1d" }
    );

    console.log("Decoded access token:", jwt.decode(accessToken));

    console.log("backend token", accessToken);

    // Set cookie for access token
    res.cookie("jwt", accessToken, {
      httpOnly: false,
      secure: false, // Set to true when running over HTTPS
      sameSite: "Lax", // Allow cross-site requests
      maxAge: 2 * 60 * 60 * 1000, // 2 hours
      domain: "localhost", // Set domain to localhost
      path: "/", // Set path to root
    });

    // Send response with both tokens
    res.json({ accessToken, refreshToken });
  } catch (error) {
    console.error("Error during login:", error);
    res.status(500).json({ message: "Internal server error" });
  }
});
```

Figures 52 & 53 Log In API

```
import nodemailer from "nodemailer";  207.3k (gzipped: 56.4k)
import express from "express";

// Create Express app
const app = express();

// Create nodemailer transporter
const transporter = nodemailer.createTransport({
service: "gmail",
  auth: {
    user: "youssefahmad1973@gmail.com",
    pass: "trcm dakc ezvo ddeh",
  },
  debug: true,
});

// Function to send email
const sendEmail = (receiver, message, subject, callback) => {
  // Define email template
  const htmlTemplate = `
        <h1>Email Notification</h1>
        <p>${message}</p>
    `;

  // Define email options
  const mailOptions = {
    from: "youssefahmad1973@gmail.com",
    to: receiver,
    subject: subject,
    html: htmlTemplate,
  };
```

```
  // Send email
  transporter.sendMail(mailOptions, callback);
};

// Example route handler to send email
app.post("/send-email", (req, res) => {
  const { receiver, message, subject } = req.body;

  sendEmail(receiver, message, subject, (error, info) => {
    if (error) {
      console.error("Error sending email:", error);
      res.status(500).json({ error: "Failed to send email" });
    } else {
      console.log("Email sent:", info.response);
      res.status(200).json({ message: "Email sent successfully" });
    }
  });
});

// Start the server
const port = process.env.PORT || 3002;
app.listen(port, () => {
  console.log(`Email Server is running on port ${port}`);
});

export default sendEmail;
```

Figures 54 & 55 Send Mail API

That is the summary of my project structure of the application and example of some of my code and components that was used to make the website and below are figures from my website

Model implementation

1. Data Preparation
2. Encode & Scale features
3. Prepare Data for LSTM
4. Build Model with K-Fold
5. Apply early stoppers
6. Evaluate Model

- **Data Preparation** it consisted of multiple parts first part being Handling is making visualizations between each feature to find the importance of these features when predicting the total sales

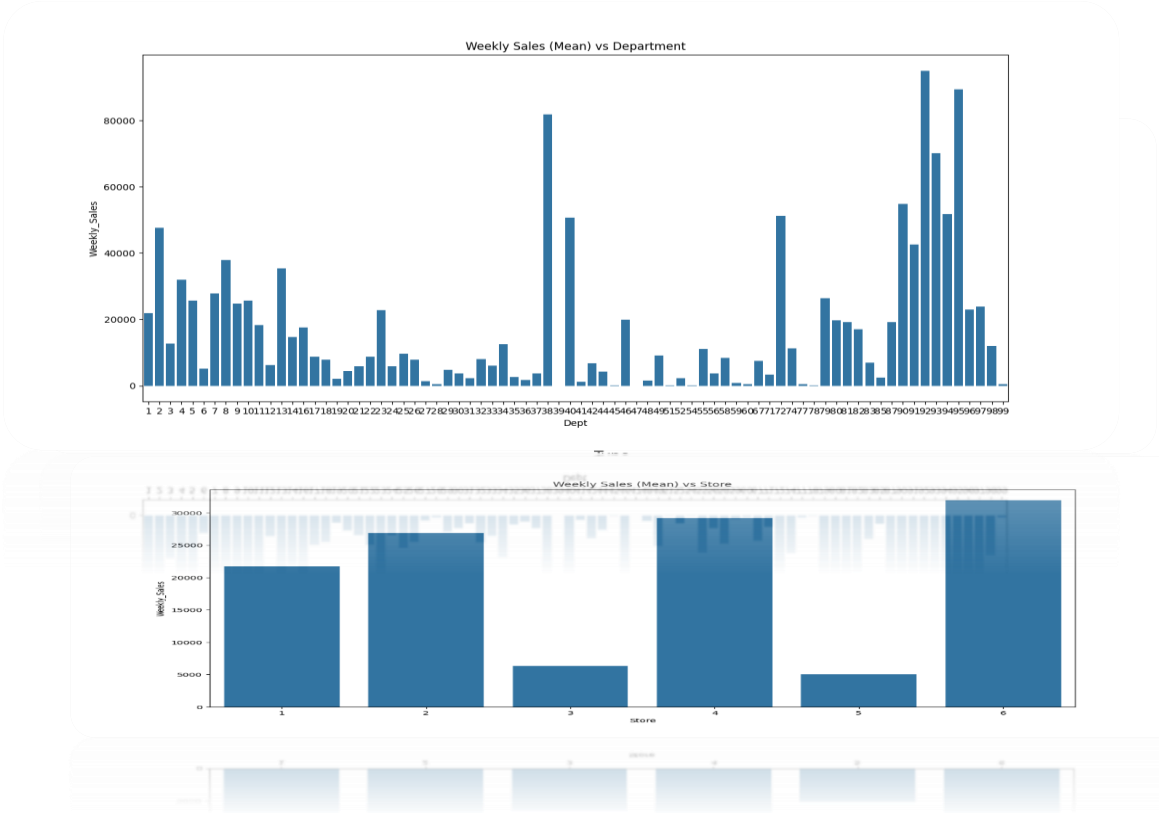
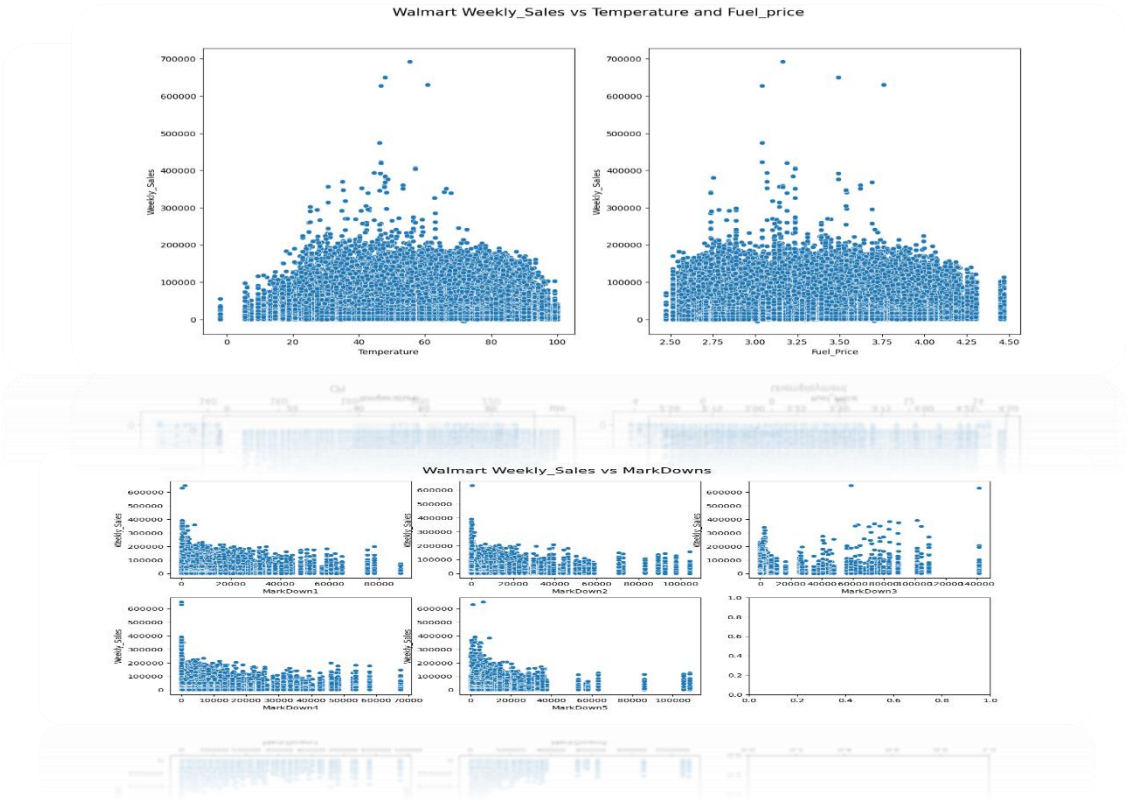


Figure 56 & 57 Figures of Features Importance

- In figures 56, and 57 we can see a noticeable variation of data in comparison with mean of weekly sales so the model will be primary based on these features so these features will be kept in the data set unchanged



Figures 58, and 59 more figures of feature importance

- **In these figures** we can see the relation between features (CPI, Unemployment, Mark Downs, Fuel Price and Temperature) it is important that the changes in the variance compared to the weekly sales was not big enough for us to consider these features in our application therefor we dropped these features from the data set so our model will only care about the most important features such as Store, Department, and Date these will be the features to be considered
- **Then the dataset** has been merged with other data set to train and test the model on all the features we merged both the test and train test with the features Dataset. Then we converted date to date time object to Data Time Objects
- **Data Time Objects** python module that allows us to work with Date and Time and makes it easier to manipulate the date as like this we are making changes to Objects we are not making changes to a string or a timestamp

❖ **Encode and scale features and handle missing values**

1. **Missing values** was imputed as it helps in cleaning the data and it will remove possible Disturbance to the model that can negatively affect the model performance, default Knn imputer with used to make sure the missing values was replaced by values that are close to it to make sure our data set stay consistent

2. **Data types** other than a number the model will not be able to understand it and these data cannot be used with the model to train Data sets in figure 63 we can see how the features was encoded; isHoliday which is a Boolean have been converted from true or false to 0 or 1 and the type have been encoded with 0,1 or 2 using label_encoder
3. **Then the features were scaled** it is known that each feature have scale to it which in some cases these features have a high weightage which will make the model biased for features more than others which in the end will make the model have unwanted behaviors, so the data have been scaled using MinMaxScaller
4. **After the preprocessing of the data:** data was then transformed into pivot table where the date is the row and store departments will be the columns this approach is used to make the data better to do time series analysis.
5. **Threshold** is defined to be used to make a different between training and testing data sets it is the point that we transition from historical data to model validation data, after that the train set will have data right after the threshold date and the test will be before it

❖ **Data scaling**

1. **Normalize data:** using min max scaler it was used to fit model data to be fixed in a range between 0 and 1 so data is easier for the model to learn and have better accuracy results

Model Definition and training:

1. **Timeseries Generator** is used to help train a model to be validated with time series by feeding them into your neural network model as you see in the figure below

```
# Generating the table
length = 48
batch_size = 20  # Number of timeseries samples in each batch
generator = TimeseriesGenerator(X_train_sc, X_train_sc, length=length, batch_size=batch_size)
```

Figure Time series generator

Length: the length of the sample used to train the model with we have it at 16 so the model needs a sequence of 16 days with consistent sales numbers to be trained to make a prediction for the week after that
Batch size: number of smaller datasets used to train each part with

2. **Lstm model** have been chosen it have 2 layers using tanh layers the first one is 64 layers and the second is 128 layers, after the data has been through the first layer it will be sent to the second layer then became the input of the dense layer; predictions are sent out to the dense layer

```
# define model
model = Sequential()
model.add(LSTM(128, activation='tanh', return_sequences=True, input_shape=(length, X_train_sc.shape[1])))
model.add(LSTM(64, activation='tanh', return_sequences=False))
model.add(Dense(X_train_sc.shape[1]))
model.compile(optimizer='adam', loss='mse')
```

Figure Model Architecture

Random forest model implementation: using the same data preprocessing used for the LSTM model that pre processed data will be used to train a random forest regression model , the Rf model created with max_samples of 0.4 and n_etsimators equal to 100 as you see in the figure below

```
# creating the Model
clf = RandomForestRegressor(n_estimators=100, max_samples = 0.4)
clf.fit(X_train_sc, y_train)
clf_pred = clf.predict(X_test_sc)

print('Results with NO differentiation')
print('Weekly_Sales_Mean_test:' ,round(np.mean(y_test) ))
print('Root Mean Squared Error:', round(np.sqrt(metrics.mean_squared_error(y_test, clf_pred))))
print('R2 Score:', r2_score(y_test, clf_pred))
```

Figure Rf model architecture

And in the figure below it displays the evaluation metrices of the model RMSE and R-squared was used to evaluate the model

```
memory usage: 57.1+ MB
Results with NO differentiation
Weekly_Sales_Mean_test: 15843
Root Mean Squared Error: 3328
R2 Score: 0.9774567631321174
```

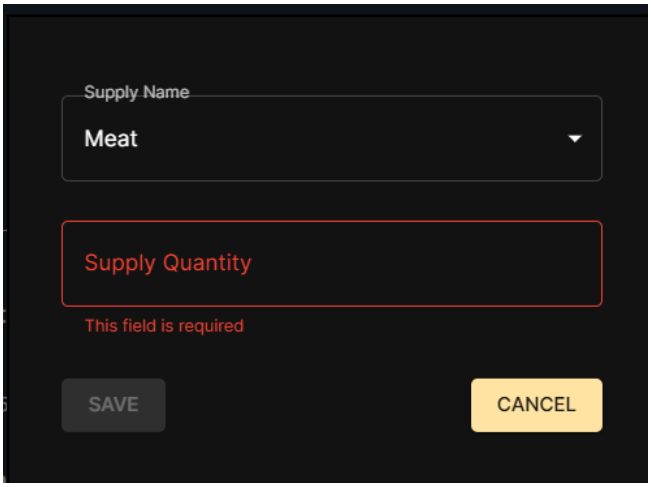
Figure Rf evaluation

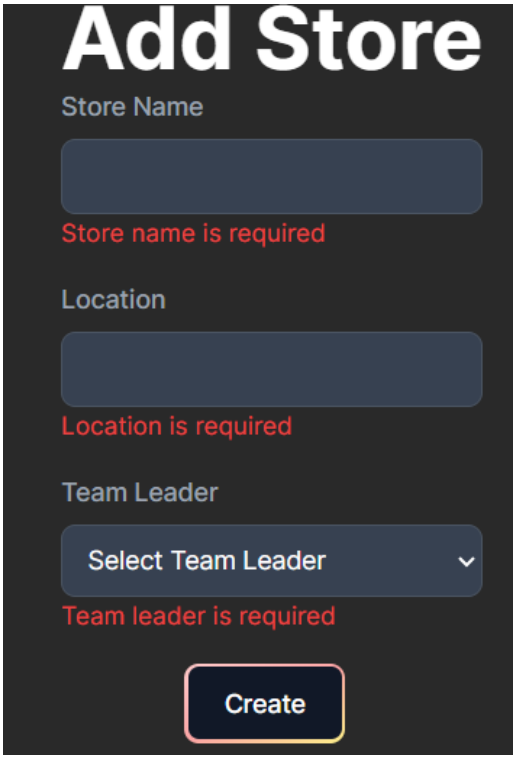
6. Testing and evaluation

a. Testing

The following test metrices was used in the Application implementation was making sure that the forms are all validated, all confirmation dialogs are displayed properly as this as important feature in the application inputting data and comparing it with the expected outcomes; big portion of the application is handling user inputs and saving it to be used later

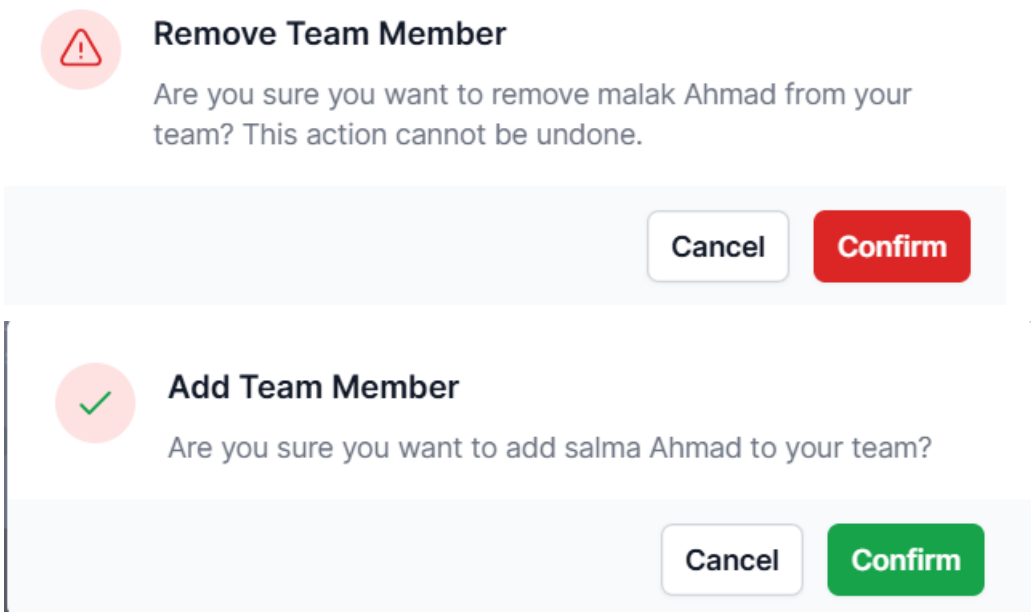
❖ Form validations





Figures of form validation

- Validation is working as it should if a field is required in a form the user will be notified of the missing field and will not be able to submit the form until he fills the empty required field



Figures of dialog

- **Dialogs** is an important part of an application like this because you can easily delete something and then forget about it or the action have been already done and that record have been deleted and it may not be possible to get back the data, same goes in the case of adding a record you may not want to add that record and add it without being aware of the record addition. So dialog will appear before making the request to the back end of the application.

b. Evaluation

- **Model Evaluation**

The following score metrices have been applied to the model: RMSE, , R-squared

1. **Before making evaluation** for the model, it must predict sales by making it make prediction for the next time series by giving it a time series just before it
2. **Then because** the data was scaled to be fed into the model to make the model capable of learning efficiently the predictions in the model will be scaled back that we can analyze the data for results

```
# Making predictions
n_features = X_train_sc.shape[1]
test_predictions = []
first_eval_batch = X_train_sc[:length]
current_batch = first_eval_batch.reshape((1, length, n_features))

for i in range(len(test_rnn)):
    # get prediction 1 time stamp ahead ([0] is for grabbing just the number instead of [array])
    current_pred = model.predict(current_batch)[0]
    # store prediction
    test_predictions.append(current_pred)
    # update batch to now include prediction and drop first value
    current_batch = np.append(current_batch[:, 1:, :], [[current_pred]], axis=1)

# Creating prediction df
inv_test_pred = mms.inverse_transform(test_predictions)
pred_df = pd.DataFrame(data=inv_test_pred, columns=test_rnn.columns, index=test_rnn.index)
```

Figure of making predictions

3. **In the figure below** we can see a plot that is representing the difference between model prediction and the actual vale of the weekly sales

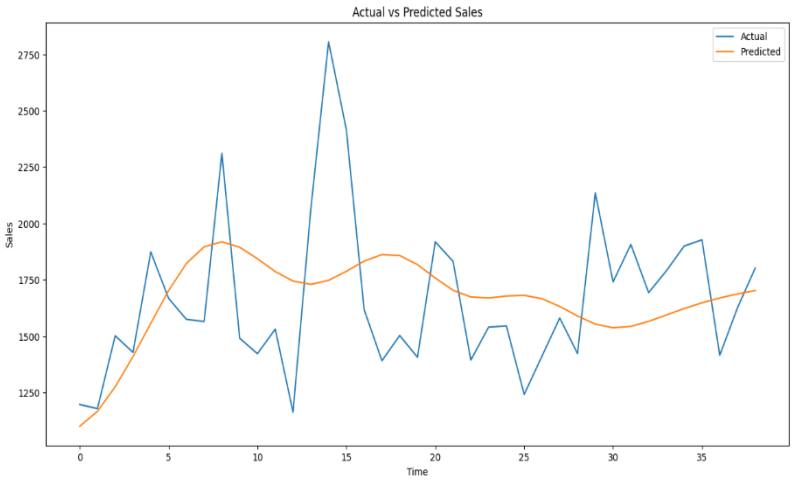


Figure actual vs predicted

4. **Model is made** to make re predictions, but it will make predictions for different stores to make sure it has variance of data to make predictions in real life and makes the model capable of dealing with more than one time series at a time
5. **Evolution metrices** like RMSE evaluations score was used by making it take the mean of actual sales numbers and the numbers predicted by the model which reached 4980 and the r-2 squared was used to get the accuracy of the model and it reached a max of 94.9 as you can see by the figure below

```
Whole process took 838 seconds
Results of RNN Model
Weekly_Sales_Mean_test: 15843
Root Mean Squared Error: 4980
R2 Score: 0.9495316398129656
Saved model architecture to 'model.js
```

Figure 60 model score

6. **Then a plot is made** for all the departments of actual vs sales predicted for store 12 as you can see in the figure below

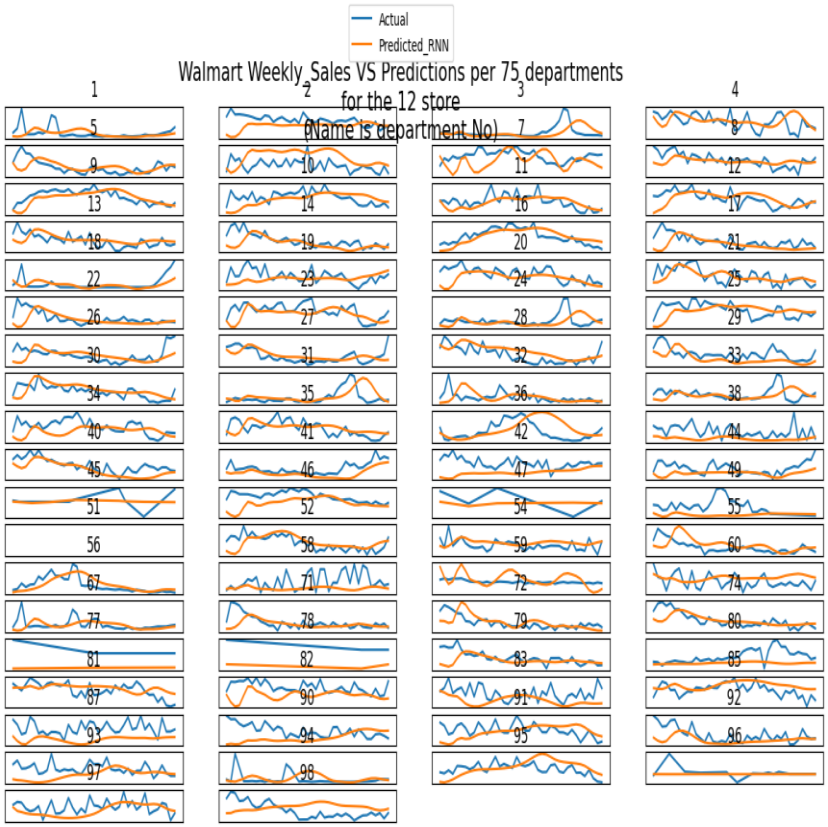


Figure 61 departments predicted vs actual

❖ **Website Evaluations**

The following survey were made to see who different people felt about the feeling of the app in ratings for items such as how easy it to use the application, how good is the design, do they think that the Application is helpful or not and other questions in the survey. The age is considered in the group so it will be for different age groups such as 20 to 35, 35 to 50 and 50 or above.

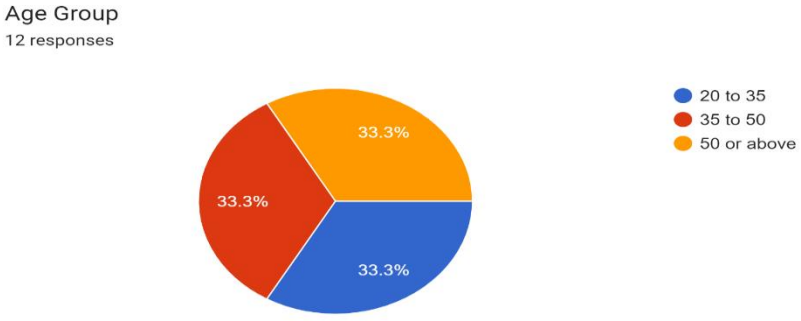


Figure Survey age group

❖ **Age Group** I have made 12 people answer a simple survey to asl them about what do they think about the software product from different age groups so we can get a variance in the responses

❖ **Design** In the figure above we can see that the design rating has been always 7 or above for 91.7 of the people who did the survey so overall they liked the design of the application

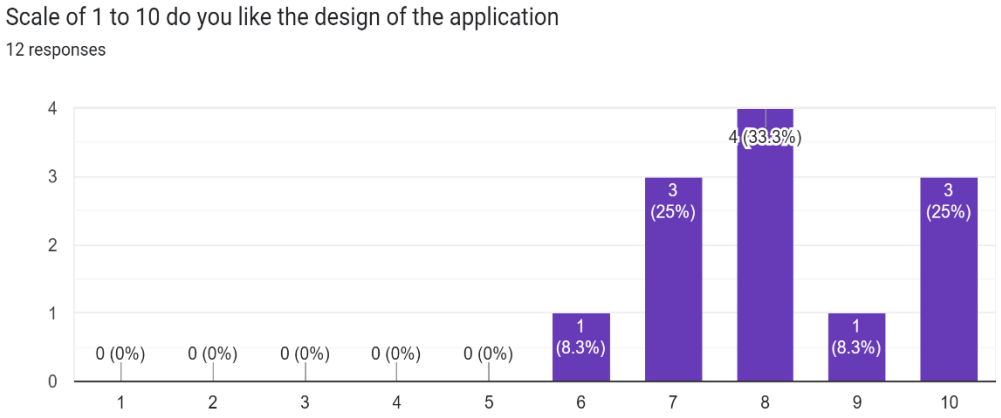


Figure Survey design

❖ **Ease of use:** we asked the participants about how easy it is use the application and the lowest rating was at 7

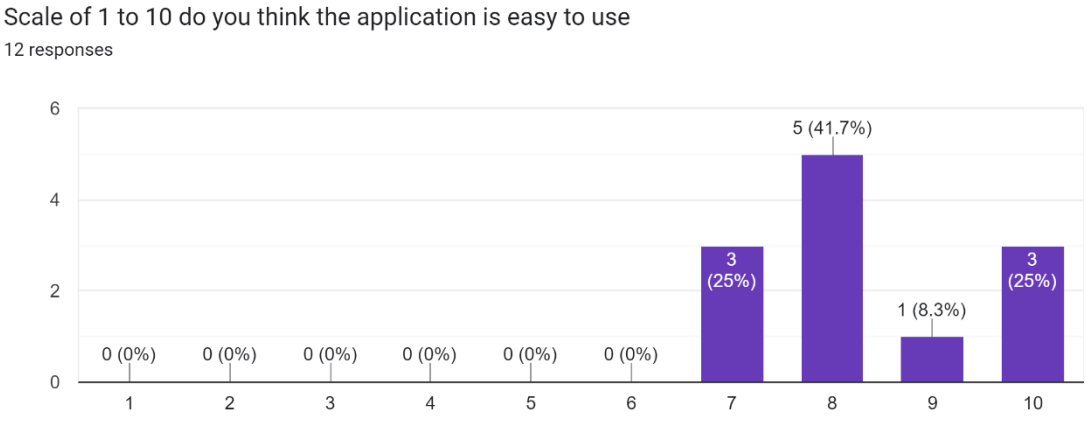


Figure survey ease of use

❖ **Application navigation:** we can see in the figure below the navigation of the application as you can see in the below figure the lowest rating was of normal which is an indicator of how easy the application is to navigate and 91.7 of the responds was easy or very easy.

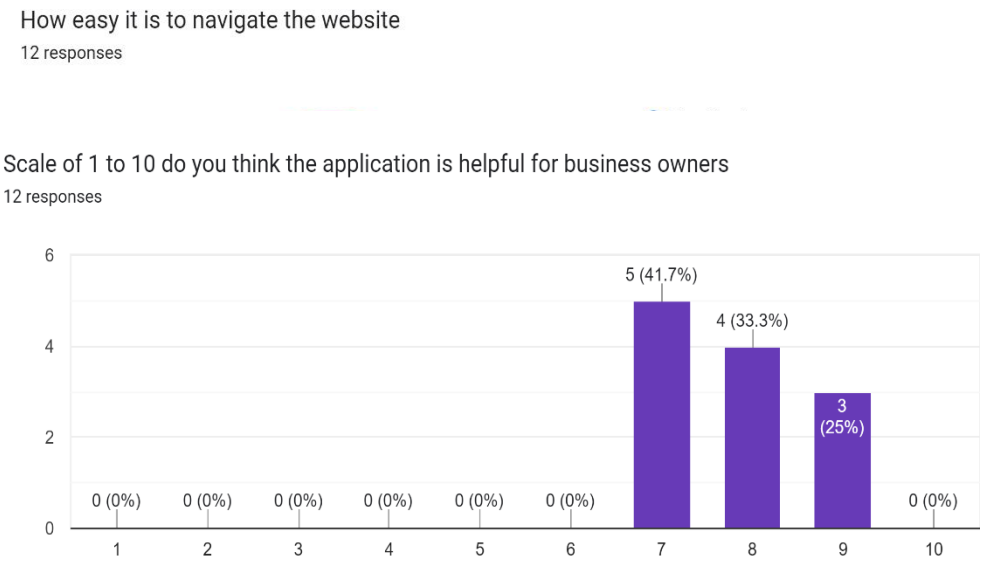


Figure Survey Navigation

- **Helpful:** we asked the participants of they thought that my application had enough functionalities to be used to manage a real business and the lowest rating we got was 7 but we did not get a rating of 10 as the application was still did not have all the needed functionalities as you can see in the figure below.

7. Results and Discussions

- **Application discussion** in the end the end software product is good for making most of business daily operations but there is still some room for improvements other features will be implemented to make this a strong and helpful tool for business owners to operate their business with a help of a software application
  - **From the survey** all the participants loved the design and the navigation of the website so these aspects of the project will be same in case of improvements the same structure can be used even with increased number of functions roles can be divided more to allow them to cover all the functions of the application
  - **The only thing I** noticed the people on the survey all of them did not find the application full of the required functions to make a business aid tool, but it did have a good amount of other business operations tasks

- **Model Discussion**
- **After trying** with models such as ARIMA, SARIMA and LSTM the LSTM made the best accuracy scores and was not as resources intensive as other models can be here in the figure below is how good the model made after training it with the data set it shows the actual vs predicted mean values and you can see the line of the predicted is near the actual values but it was only close in the short range when the time got over 10 the model performance was degraded

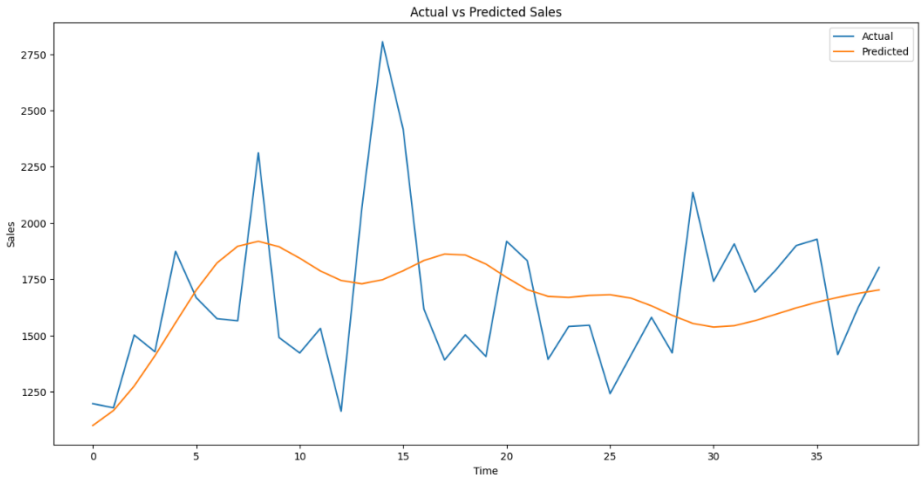


Figure 62 predicted vs actual values

- But in this figure below we can see predictions for specific store number 12 actual vs predicted values was compared with all the different departments of this store. Some departments the model predictions were really close to the actual values but that happened only on same departments

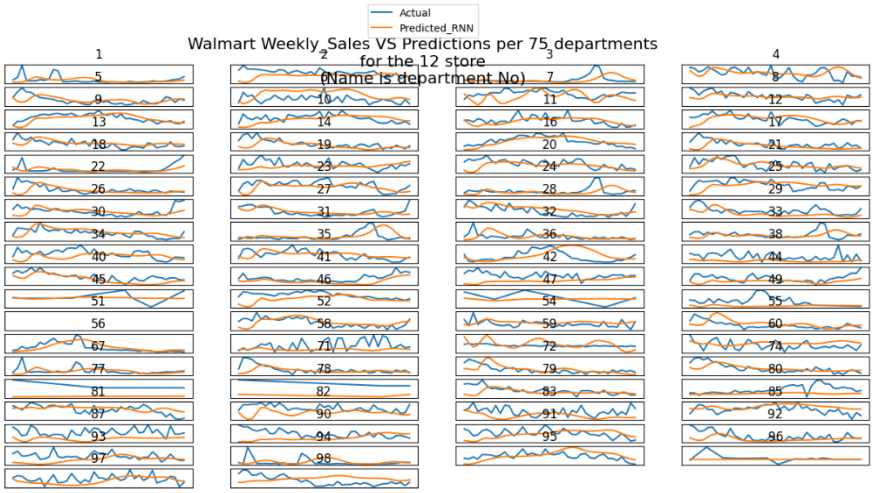
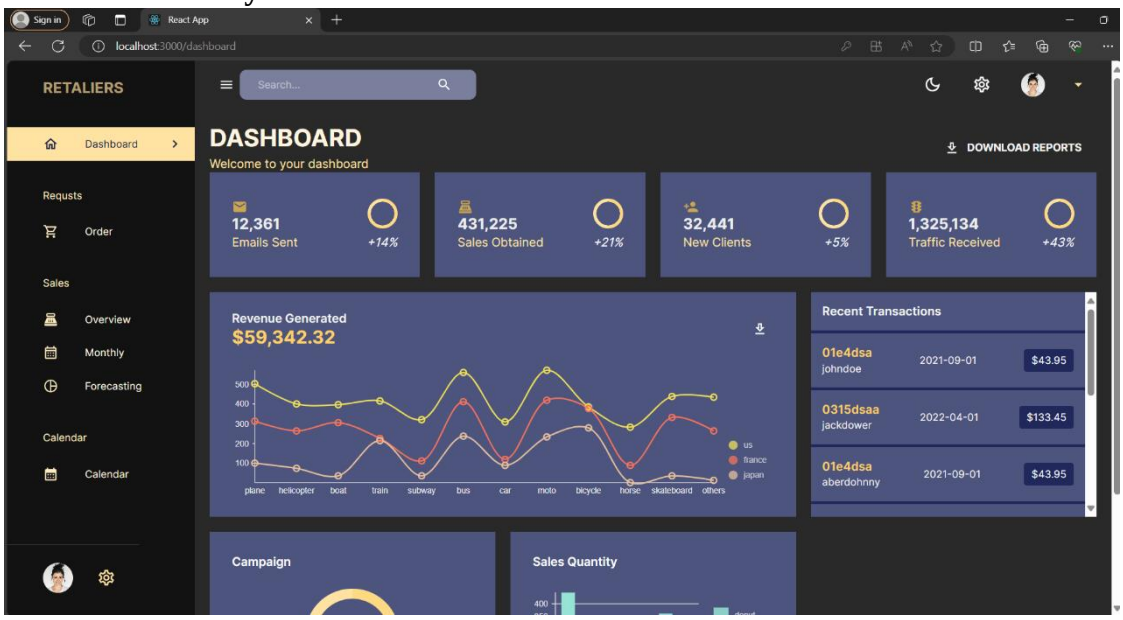


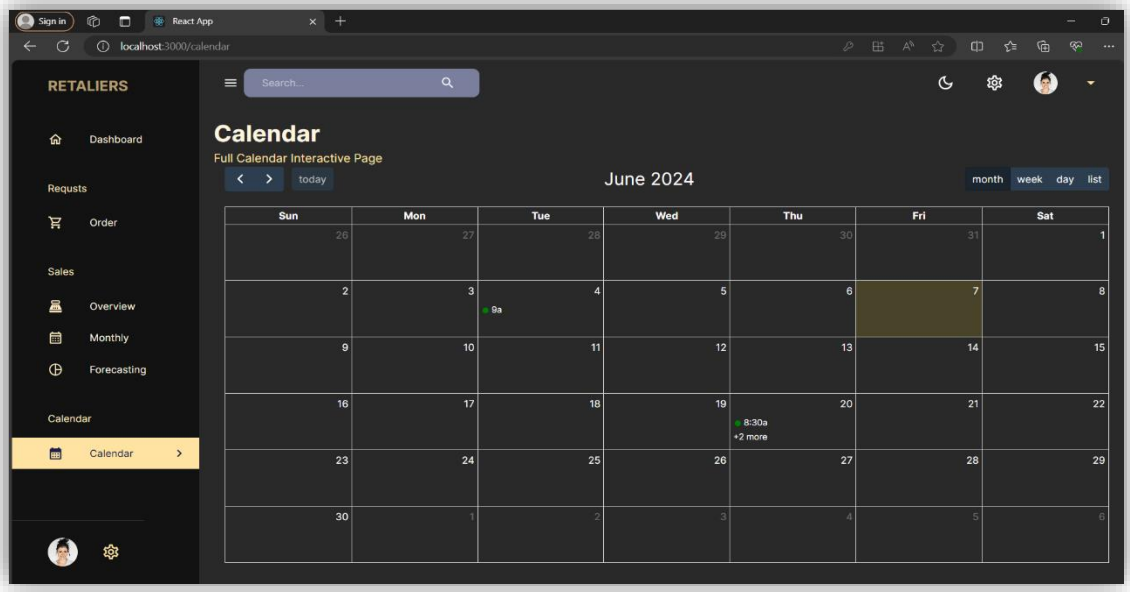
Figure 63 store 12 predictions vs actual values

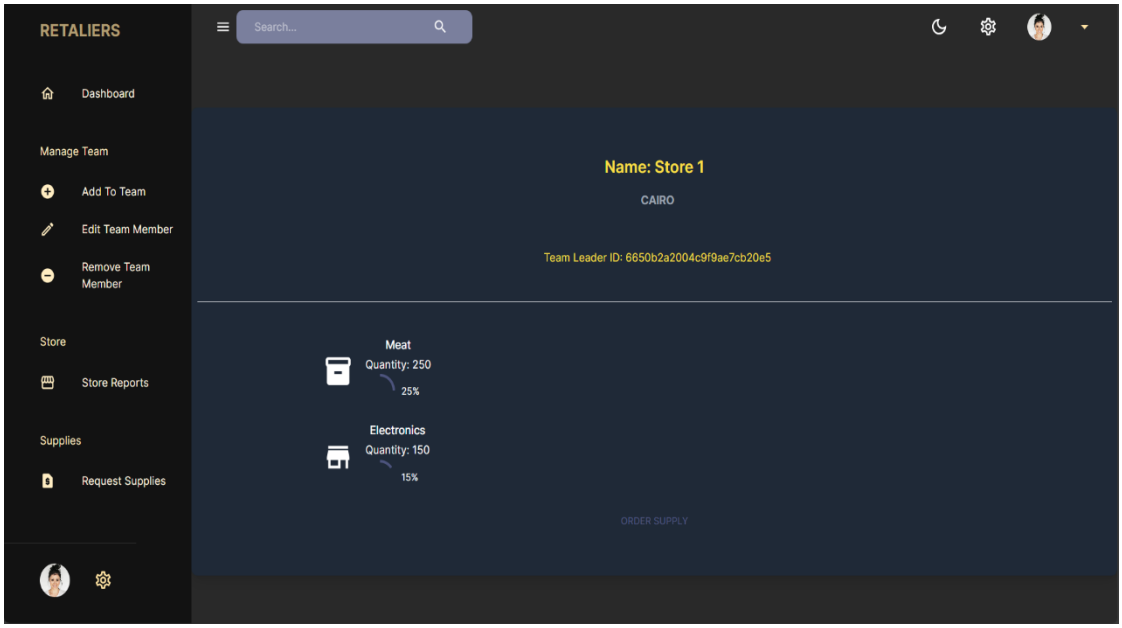
- **Training the model** with bigger and improved data set would increase model performance and make the predictions closer to the actual values of weekly sales
- **Overall model** performance and accuracy was not bad at all, and the model can be used to make helpful predictions
- **Model integration** model was easily integrated to the application using flask-cors library in python which puts a route for the python predict function and then these routes can be called from the application like a normal request that is made to the back end
- **Predict function** it was not easy to match the input size of the user input with what should be the input of the model that input that was used to train the data set, so I was not able to fully implement the prediction's function

8. Conclusions and Future Work

a. Summary







Figures 64, 65 and 66 example of pages

- **in the end** I was able to develop a website based on MERN stack that should be fully equipped to be used to provide business owners with a tool that helps them to manage their day-to-day operations.
- **with the help** of neural network models such as Lstm, Arima and random forest that was used to make the final model; it peeked r-squared accuracy scores at 94,9 as you can see on the following figure below

```
Whole process took 838 seconds
Results of RNN Model
Weekly_Sales_Mean_test: 15843
Root Mean Squared Error: 4980
R2 Score: 0.9495316398129656
Saved model architecture to 'model.js
```

Figure Model scores

- **Good Model results** came from the model training and testing that if integrated with another data set that have more importance in the predicting of sales that would make an amazing business tools that can still be improved or increased
- **Integrated model** can still be trained to be an ensemble models by taking the model after being trained on the original data set and train it again using Random Forest model or any regression model with multivariances with added and improved model predictions

b. Future Work

Explain any limitations in your results and how things might be improved. Discuss how your work might be developed further. Reflect on your results in isolation and in relation to what others have achieved in the same field. This self-analysis is particularly important. You should give a critical evaluation of what went well, and what might be improved.

Points of Improvements

- Application is only designed to be used with a laptop or a computer, it is not responsive enough to be used with other devices such as tablets and smartphones.
- Model is not able to make predictions of sales as there was issues found in implementation of that par; the model was easily integrated with the application but matching the user input with the model required input to make predictions was not an easy part in making the predictions functions. So future work in this area will be finding a way to preprocess user data and making it compatible for the website to be used with the model
- Not all Ui components are designed to be used with different objects and classes which lead to some redundancy in the code, so in future work the Ui components will be designed to be flexible and reusable
- Model training took long time and was resources intensive

Good points of the Application

- **Authentication implementation of JWT token**: application is built with authentication in mind. To make sure the user felt safe, user passwords are all hashed so even if an attack happened to the system, it will be extremely difficult to switch the saved hashed password into plain text. Using tokens on the website is a good practice as it makes less request to the database to get important data as it is saved in the token.
- **Good user interface:** the user interface of the application is modern good looking, and it is using most popular UI libraries such as Mu (Material Ui), Tailwind and react
- **Form validation**: all the forms of the website have been validated to make sure it is the required input we are getting and that all the inputs are correct, and requests made to the back end will be less as only the correct form input will be send to the user
- **Notifications feature:** An API was implemented using NodeMailer library to make the website have notifications features by sending them by email it is a flexible and reusable function can be used with any aspect of the application and the subject and email text is also sent to the function making it easy to use and it does not require any modifications

- **Visualization techniques:** the application is providing user with dashboard to display relevant information about the business, supplies and reports are visualized with modern design and it shows the user all the important details that can help him make am important business decisions

- **Model Accuracy:** although I had issues with making the predicting function work, this does not change the scores of the model as its accuracy peeked at 94.9%

References

1. Anis, M., Chawky, S., Abdel Halim, A. (2023). Retail. In: Mapping Innovation. Springer, Cham. https://doi.org/10.1007/978-3-030-93627-3_12
2. R. Fildes, S. Ma, S. KolassaRetail forecasting: research and practice International Journal of Forecasting (2020) in press
3. Bu Ma, S. and Fildes, R., 2021. Retail sales forecasting with meta-learning. *European Journal of Operational Research*, *288*(1), pp.111-128.
4. D.H. Wolpert, W.G. Macready No free lunch theorems for optimization
   IEEE Transactions on Evolutionary Computation, 1 (1) (1997), pp. 67-82, 10.1109/4235.585893
5. Sand S. Makridakis, F. Petropoulos The M4 competition: Conclusions
   International journal of forecasting, 36 (1) (2020),
   pp. 224-227, 10.1016/j.ijforecast.2019.05.006
6. Taehooie R. Fildes, F. Petropoulos Simple versus complex selection rules for forecasting many time series
   Journal of Business Research, 68 (8) (2015), pp. 1692-1701, 10.1016/j.jbusres.2015.03.028
7. Öndas, V. (2021). A Study on High-tech Startup Failure: Antecedents, Outcome and Context.
8. Jafar Alzubi et al 2018 J. Phys.: Conf. Ser. 1142 012012
9. James G Witten D Hastie T Tibshirani R, eds. An Introduction to Statistical Learning: With Applications in R. New York: Springer; 2013.
10. Hastie T, Tibshirani R, Friedman JH. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2nd ed. New York, NY: Springer; 2009.
11. Hebb DO. Animal and physiological psychology. *Annu Rev Psychol*. 1950; 1: 173–188. doi:10.1146/annurev.ps.01.020150.001133. [CrossRef] [PubMed]
12. Rumelhart DE, Hinton GE, Williams RJ. Learning representations by back-propagating errors. *Nature*. 1986; 323: 533–536. doi:10.1038/323533a0. [CrossRef]
13. Orbach J. Principles of neurodynamics. Perceptrons and the theory of brain mechanisms. *Arch Gen Psychiatry*. 1962; 7: 218–219. doi:10.1001/archpsyc.1962.01720030064010. [CrossRef]
14. LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature*. 2015; 521: 436–444. doi:10.1038/nature14539. [CrossRef] [PubMed]
15. Lulu Wen, Kaile Zhou, Shanlin Yang, Load demand forecasting of residential buildings using a deep learning model, Electric Power Systems Research, Volume 179,2020,106073,ISSN 0378-7796, https://doi.org/10.1016/j.epsr.2019.106073 (https://www.sciencedirect.com/science/article/pii/S037877961930392X)
16. Massimo Pacella, Gabriele Papadia, Evaluation of deep learning with long short-term memory networks for time series forecasting in supply chain management, Procedia CIRP, Volume 99, 2021, Pages 604-609, ISSN 2212-8271, https://doi.org/10.1016/j.procir.2021.03.081. (https://www.sciencedirect.com/science/article/pii/S2212827121003711)

17. Sushil Punia, Surya P. Singh, Jitendra K. Madaan, A cross-temporal hierarchical framework and deep learning for supply chain forecasting, Computers & Industrial Engineering, Volume 149, 2020, 106796, ISSN 0360-8352, https://doi.org/10.1016/j.cie.2020.106796. (https://www.sciencedirect.com/science/article/pii/S0360835220305040)
18. Hossein Abbasimehr, Mostafa Shabani, Mohsen Yousefi, An optimized model using LSTM network for demand forecasting, Computers & Industrial Engineering, Volume 143, 2020, 106435,
ISSN 0360-8352, https://doi.org/10.1016/j.cie.2020.106435 (https://www.sciencedirect.com/science/article/pii/S0360835220301698)
19. Raheel Siddiqui, Muhammad Azmat, Shehzad Ahmed & Sebastian Kummer (2022) A hybrid demand forecasting model for greater forecasting accuracy: the case of the pharmaceutical industry, Supply Chain Forum: An International Journal, 23:2, 124-134, DOI: 10.1080/16258312.2021.1967081
20. Bo Zhang, Ming-Lang Tseng, Lili Qi, Yuehong Guo, Ching-Hsin Wang,
A comparative online sales forecasting analysis: Data mining techniques, Computers & Industrial Engineering, Volume 176, 2023, 108935, ISSN 0360-8352, https://doi.org/10.1016/j.cie.2022.108935. (https://www.sciencedirect.com/science/article/pii/S0360835222009238)
21. A.L.D. Loureiro, V.L. Miguéis, Lucas F.M. da Silva, Exploring the use of deep neural networks for sales forecasting in fashion retail, Decision Support Systems, Volume 114, 2018, Pages 81-93, ISSN 0167-9236, https://doi.org/10.1016/j.dss.2018.08.010. (https://www.sciencedirect.com/science/article/pii/S0167923618301398)

22. T. Chiu, D. P. Fang, J. Chen, Y. Wang, and C. Jeris, “A robust and scalable clustering algorithm for mixed type attributes in a large database environment,” in *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, San Francisco, CA, USA, August 2001.

View at: Publisher Site | Google Scholar

23. T. Zhang, R. Ramakrishnan, and M. Livny, “Birch: a new data clustering algorithm and its applications,” *Data Mining and Knowledge Discovery*, vol. 1, no. 2, pp. 141–182, 1997.
View at: Publisher Site | Google Scholar
24. Shouwen Ji, Xiaojing Wang, Wenpeng Zhao, Dong Guo, "An Application of a Three-Stage XGBoost-Based Model to Sales Forecasting of a Cross-Border E-Commerce Enterprise",

Mathematical Problems in Engineering, vol. 2019, Article ID 8503252, 15 pages, 2019. https://doi.org/10.1155/2019/8503252

25. He Wei and QingTao Zeng 2021 J. Phys.: Conf. Ser. 1754 012191